



4장.분류-Part1(4.1~4.4)

3팀-권혜수, 조한희, 이시현

목차

#4.1 분류(Classification)의 개요

#4.2 결정 트리

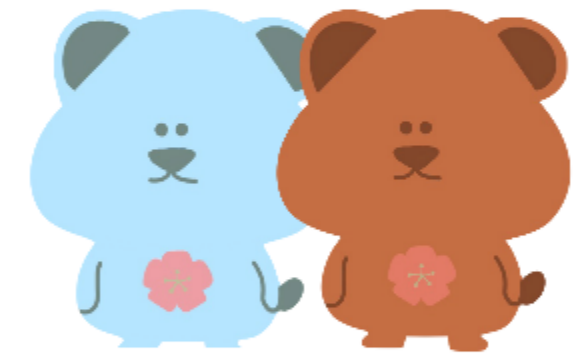
#4.3 앙상블 학습

#4.4 랜덤포레스트

#5.1 SVM



4.1 분류(Classification)의 개요



#1 분류(Classification)의 개요

#1 지도학습 : 레이블(Label), 즉 명시적인 정답이 있는 데이터가 주어진 상태에서 학습하는 머신러닝 방식.

→ 지도 학습 대표적 유형인 분류(Classification) : 학습 데이터(데이터의 피쳐 & 레이블값)을 머신러닝 알고리즘으로 학습해 모델을 생성하고, 새로운 데이터 값으로 미지의 레이블 값을 예측

#2 분류를 구현하는 머신러닝 알고리즘

- 나이브 베이즈(Naive Bayes) : 베이즈(Bayes) 통계와 생성 모델 기반
- 로지스틱 회귀(Logistic Regression) : 독립변수와 종속변수의 선형 관계성 기반
- 결정 트리(Decision Tree): 데이터 균일도에 따른 규칙 기반
- 서포트 벡터 머신(Support Vector Machine): 개별 클래스 간 최대 분류 마진 탐색
- 최소 근접(Nearest Neighbor) 알고리즘: 근접 거리 기준
- 신경망(Neural Network): 심층 연결 기반
- 앙상블(Ensemble) : 서로 다른(또는 같은) 머신러닝 알고리즘 결합

#4.1 분류(Classification)의 개요

#3 앙상블 방법(Ensemble Method)

: 정형 데이터 예측 분석 영역 주도 vs 비정형 데이터(이미지, 영상, 음성 등)은 신경망 딥러닝 주도

#4 앙상블 형태

1) 서로 다른/같은 알고리즘 단순 결합: 대부분 동일한 알고리즘 결합

2) 배깅(Bagging) 방식

ex) **랜덤 포레스트(Random Forest)**: 예측 성능, 상대적으로 빠른 수행 시간, 유연성

3) 부스팅 (Boosing) 방식

ex) **그래디언트 부스팅(Gradient Boosting)**: 뛰어난 예측 성능 수행 시간 ↑

XgBoost(eXtra Gradient Boost)와 LightGBM에서 수행 시간 단축하며 활용도 1위

#5 앙상블 기본 알고리즘: 결정 트리

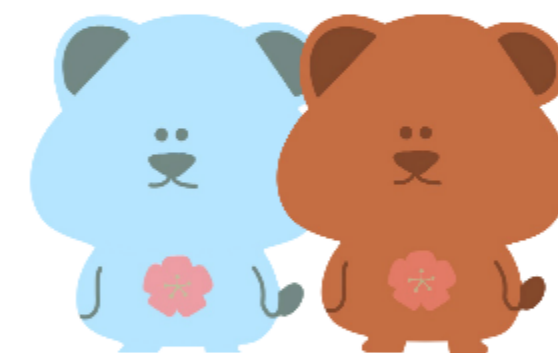
: 매우 쉽고 유연하게 적용 & 데이터 사전 가공의 영향 적음(데이터 스케일링, 정규화 등)

복잡한 규칙 구조로 인한 과적합(overfitting)으로 예측 성능 저하 가능성

➔ 오히려 장점

➔ 앙상블은 약한 학습기를 多 결합 -> 확률적 보완, 오류 발생 부분에 대한 가중치 업데이트

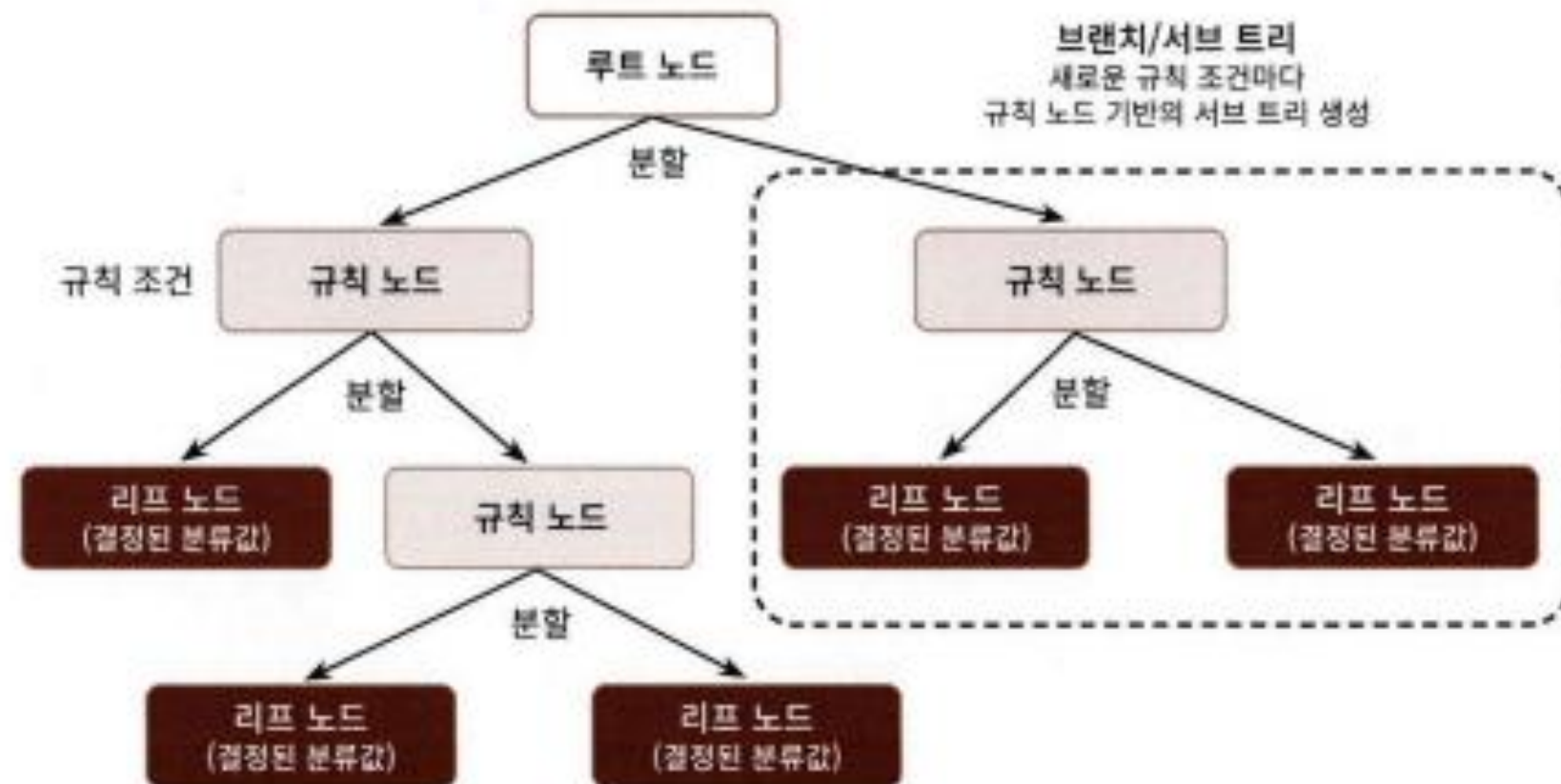
4.2 결정 트리



#0 결정 트리 (Decision Tree)

#1 직관적이고 이해하기 쉬운 알고리즘

- 데이터에 있는 규칙을 학습 & 패턴 발견 → **트리(Tree)** 기반의 분류 규칙 생성
- if, else 규칙 기반 → 규칙을 만들어내는 '**기준**' 이 효율적 분류 성능 좌우

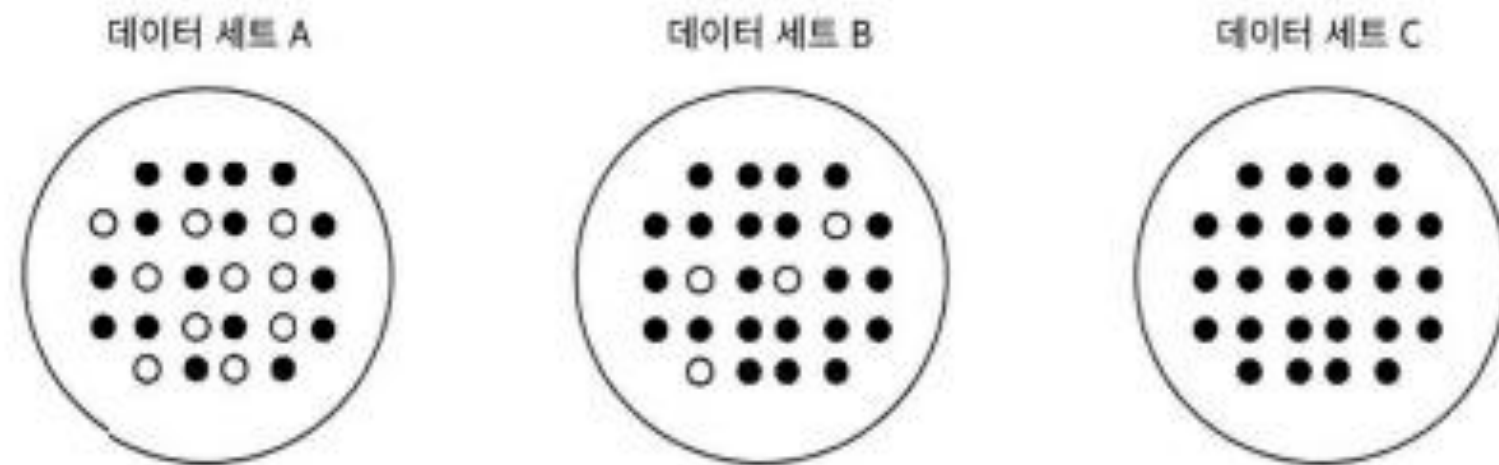


- 1) 규칙 노드(Decision Node): 규칙 조건
- 2) 리프 노드(Leaf Node): 결정된 클래스 값
- 3) 서브 트리(Sub Tree): 피쳐들의 결합으로 새로운 규칙 조건을 만들 때 마다 생성

#0 결정 트리 (Decision Tree)

#2 데이터 세트의 균일도

- 트리의 깊이(depth): 깊을수록 규칙이 많다는 의미, 즉 오버피팅의 가능성 ↑
- 분류 시 최대한 많은 데이터 세트가 해당 분류에 속할 수 있도록
= 최대한 **균일한 데이터 세트**를 구성하도록 분할(Split)



균일도 $C > B > A$

<정보 균일도가 높은 데이터 세트를 먼저 선택할 수 있도록 규칙 조건 생성>

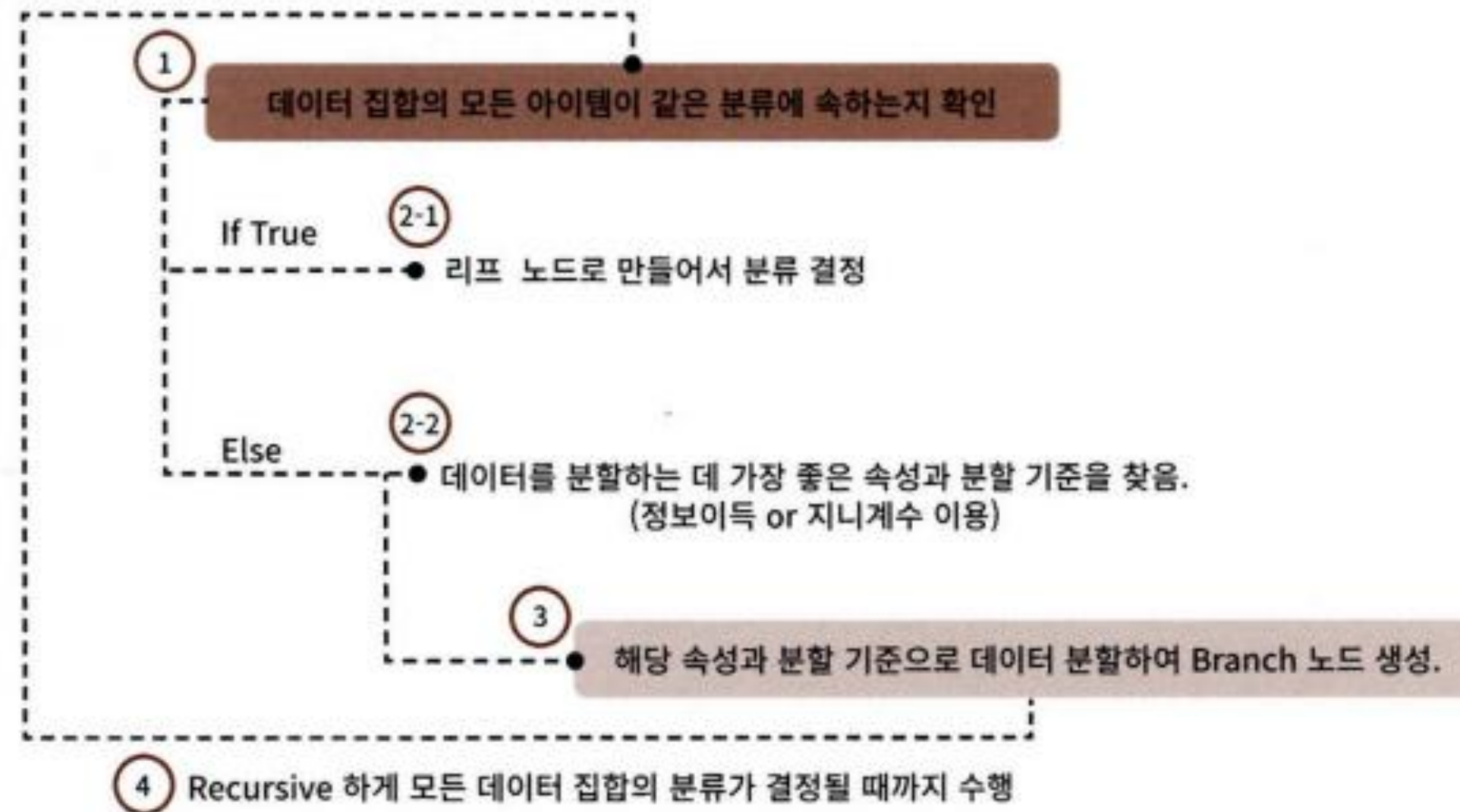
1. 정보이득(Information Gain): 높을수록 균일도가 높음, 1-엔트로피(엔트로피가 높으면 혼잡)
2. 지니 계수: 0~1, 지니계수가 낮을수록 균일도가 높음

Ex. 원형 라벨의 색상 피처가 모두 노란색, 네모 도형의 피처는 파랑과 초록이 섞여 있음
그러면 규칙은 `if color == yellow` 로 생성되고, 분할 시 해당 자식 노드는 모두 원형으로 불순도는 0이 됨

#0 결정 트리 (Decision Tree)

#3 DecisionTreeClassifier 작동 원리

- (기본) 지니 계수를 이용해 데이터 세트를 분할
- 데이터 세트를 분할 조건: 정보 이득 $\uparrow$ or 지니 계수 $\downarrow$
- 데이터가 모두 특정 분류에 속하면 분류 결정



#1 결정 트리 모델의 특징

장점	단점
쉽고 직관적 • 정보의 '균일도' 명확한 룰 • 규칙 노드, 리프 노드 생성 방식 • 시각화 전처리 작업 x (피처의 스케일링, 정규화 등)	과적합 -> 성능 하락 • 피처 多 & 균일도 다양해지면서 • 깊이 ↑ 복잡 • 트리의 크기를 사전에 제한

#2 결정 트리 파라미터

파라미터 명	설명
min samples split ()	• 노드를 분할하기 위한 최소 샘플 데이터 수로 과적합을 제어하는 데 사용 • 작게 설정 -> 분할 노드 多 -> 과적합 증가 • Default = 2
min_samples_leaf	• 말단 노드가 되기 위한 최소한의 샘플 수 • 크게 설정하면 말단 노드의 최소 샘플 데이터 수 조건 만족시키기 어려움 -> 노드 분할을 상대적으로 덜 수행함 (과적합 제어) • 비대칭적(imbalanced) 데이터 경우 작게 설정: 특정 클래스의 데이터가 극도로 작을 수 있음
max features	• 최적의 분할을 위해 고려할 최대 피쳐 개수 • Default = None으로 데이터 세트의 모든 feature를 사용하여 분할 • 'int' (대상 피쳐 개수), 'float' (전체 피쳐 중 대상 피쳐의 퍼센트%), 'sqrt'와 'auto' (전체 피쳐 중 제곱근 값), 'log' (전체 피쳐 중 $\log_2$(전체 피쳐 개수)), 'None'은 전체 피쳐 선정
max_depth	• 트리의 최대 깊이 지정 • Default = None. 이는 완벽하게 클래스 결정 값이 될 때까지 분할 & 노드의 데이터 개수가 min_samples_split보다 작아질 때까지 • 깊이가 깊어지면 min_samples_split 설정대로 최대 분할하여 과적합할 수 있으므로 적절한 값으로 제어 필요.
max_leaf_nodes	• 말단 노드(Leaf)의 최대 개수

#3 결정 트리 모델의 시각화

#1 Graphviz 패키지

- dot 파일(그래프 기반)을 쉽게 시각화할 수 있는 패키지
- graphviz.Source: DOT 언어(Graphviz의 그래프 표현 언어)로 작성된 문자열을 받아서 그래프로 보여주거나 파일로 저장
- **export_graphviz()**: Graphviz가 읽어 들어서 그래프 형태로 시각화할 수 있는 출력 파일 생성
 - 학습 완료된 Estimator, 피처의 이름 리스트, 레이블 이름 리스트
 - 학습된 결정 트리 규칙을 실제 트리 형태로 시각화

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import warnings
warnings.filterwarnings('ignore') #경고메시지를 무시하겠다는 의미
```

#학습된 DecisionTree Classifier 생성

```
dt_clf = DecisionTreeClassifier(random_state=156)

iris_data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris_data.data, iris_data.target,
                                                    test_size=0.2, random_state=11)

dt_clf.fit(X_train, y_train)
```

```
from sklearn.tree import export_graphviz
import graphviz
```

#호출 결과로 out_file로 지정된 tree.dot 파일 생성

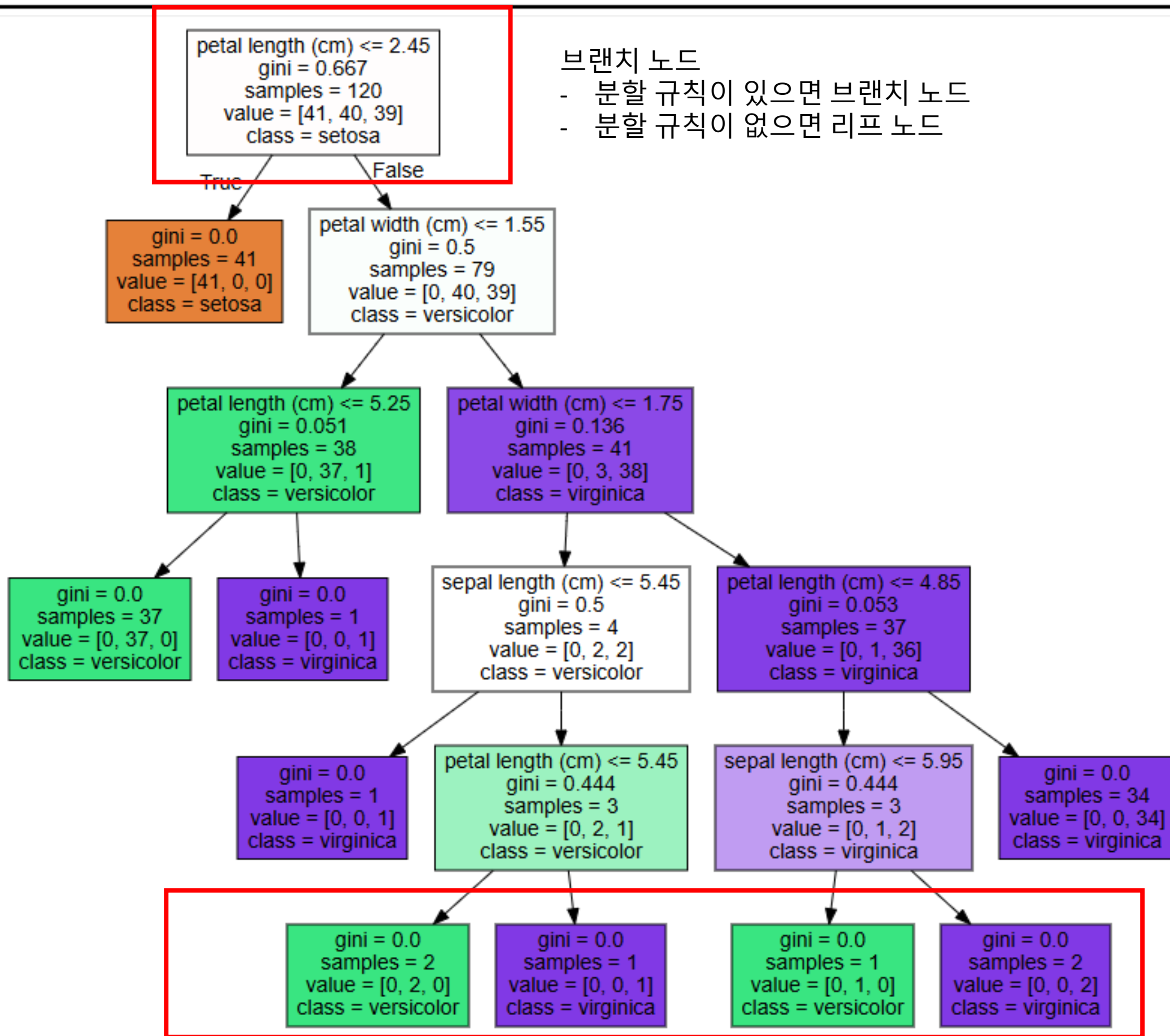
```
export_graphviz(dt_clf, out_file='tree.dot', class_names=iris_data.target_names,
                feature_names=iris_data.feature_names, impurity=True, filled=True)
```

```
#위에 생성된 tree.dot 파일을 Graphviz가 읽어서 시각화
with open('tree.dot') as f:
    dot_graph = f.read()
graphviz.Source(dot_graph)
```

*디폴트:

- Impurity=True
- Filled = False

#3 결정 트리 모델의 시각화



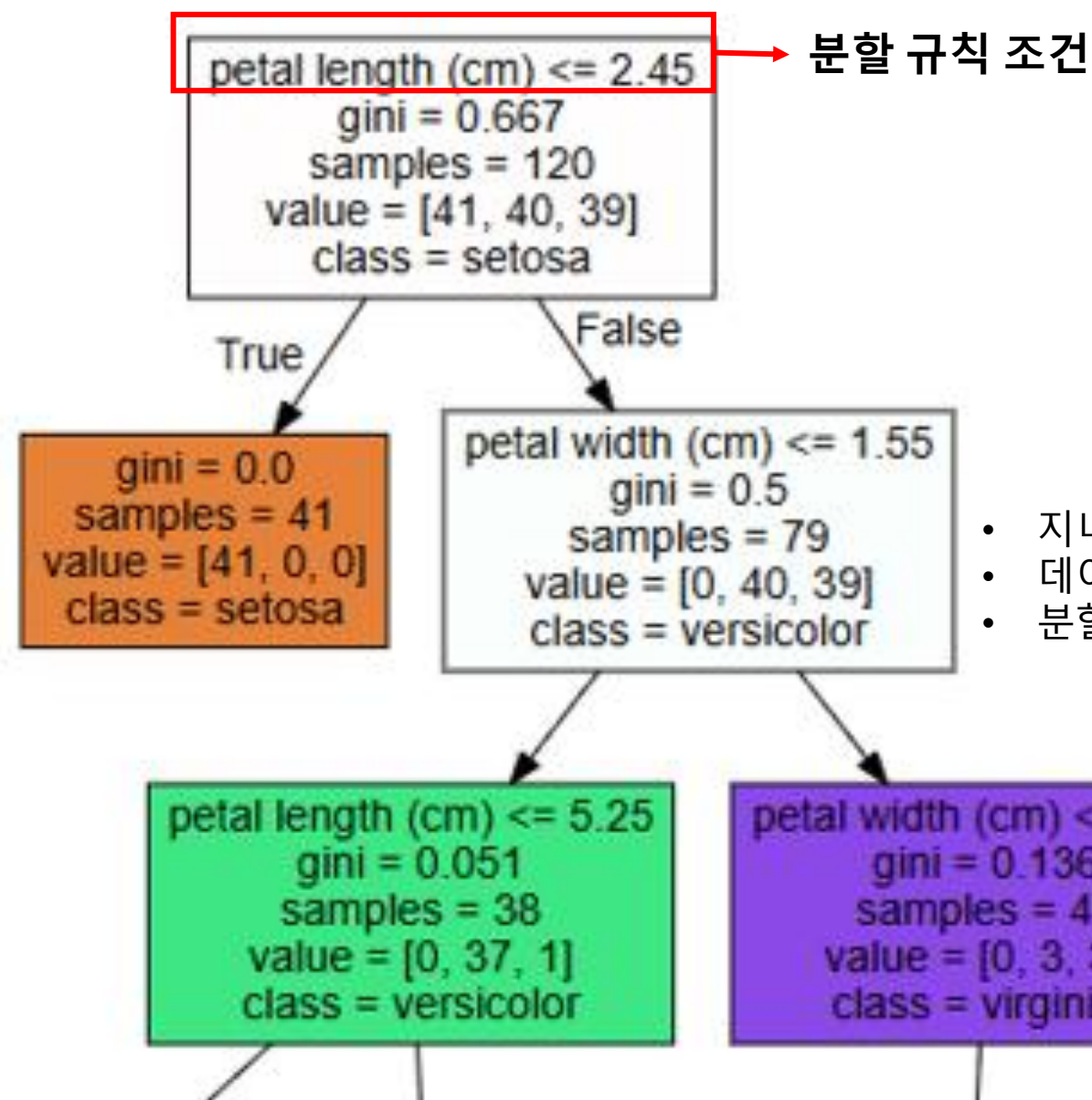
브랜치 노드

- 분할 규칙이 있으면 브랜치 노드
- 분할 규칙이 없으면 리프 노드

리프 노드

- 데이터가 오직 하나의 클래스 값
- 하이퍼 파라미터 조건 충족

#3 결정 트리 모델의 시각화



* 지표 설명

- **gini**(지니계수) `value = [ ]`로 주어진 데이터 분포
- **Samples**: 현 규칙에 해당하는 데이터 건수
- **value = []**: 클래스 값에 해당하는 데이터 건수
- **class = setosa**: setosa의 개수가 제일 많음
- **색**: 붓꽃 데이터의 레이블 값
- 색이 짙을수록 지니 계수 ↓ 레이블의 샘플 데이터 ↑

- 분할 규칙 조건 x = 리프 노드
- 41개의 모든 데이터가 setosa
- gini = 0

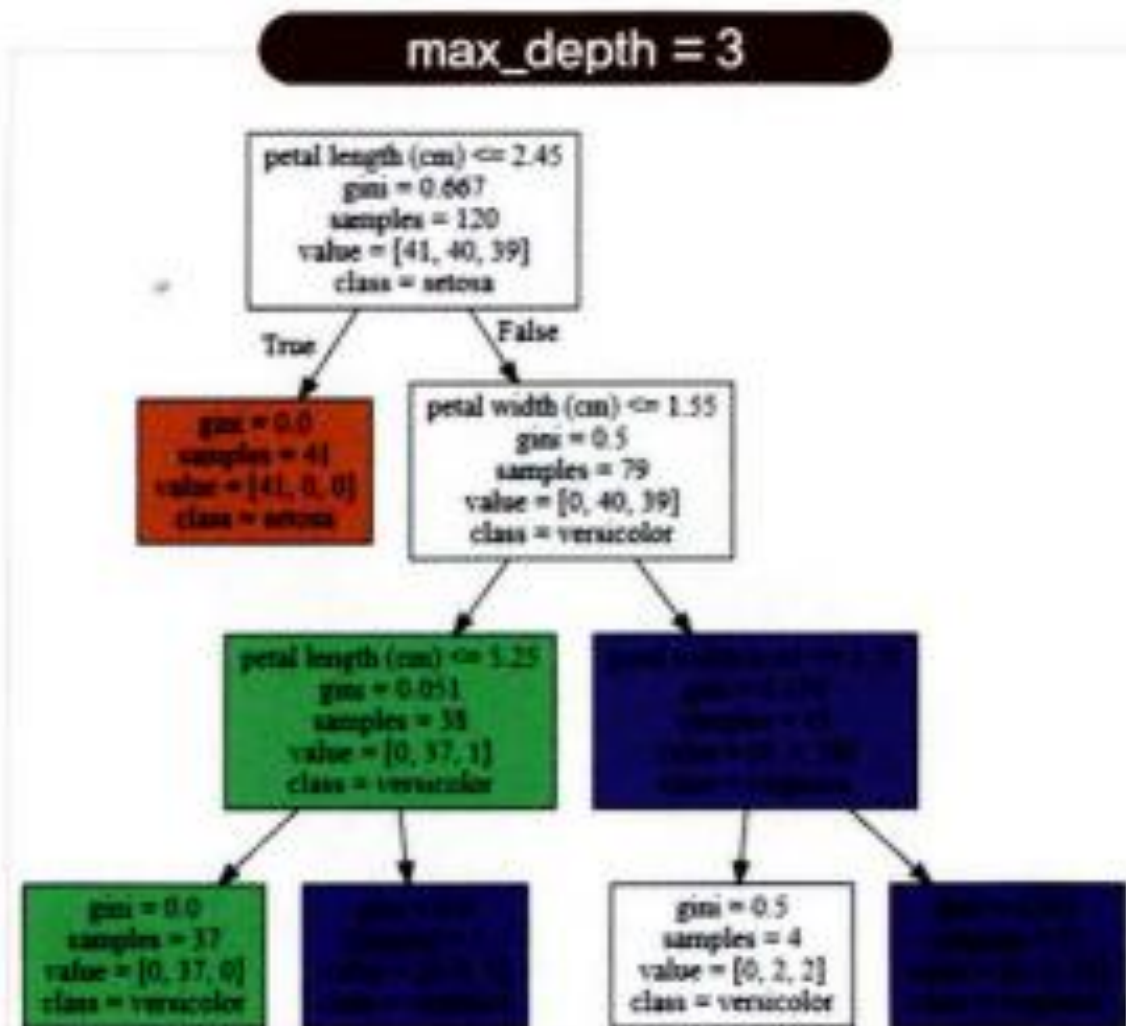
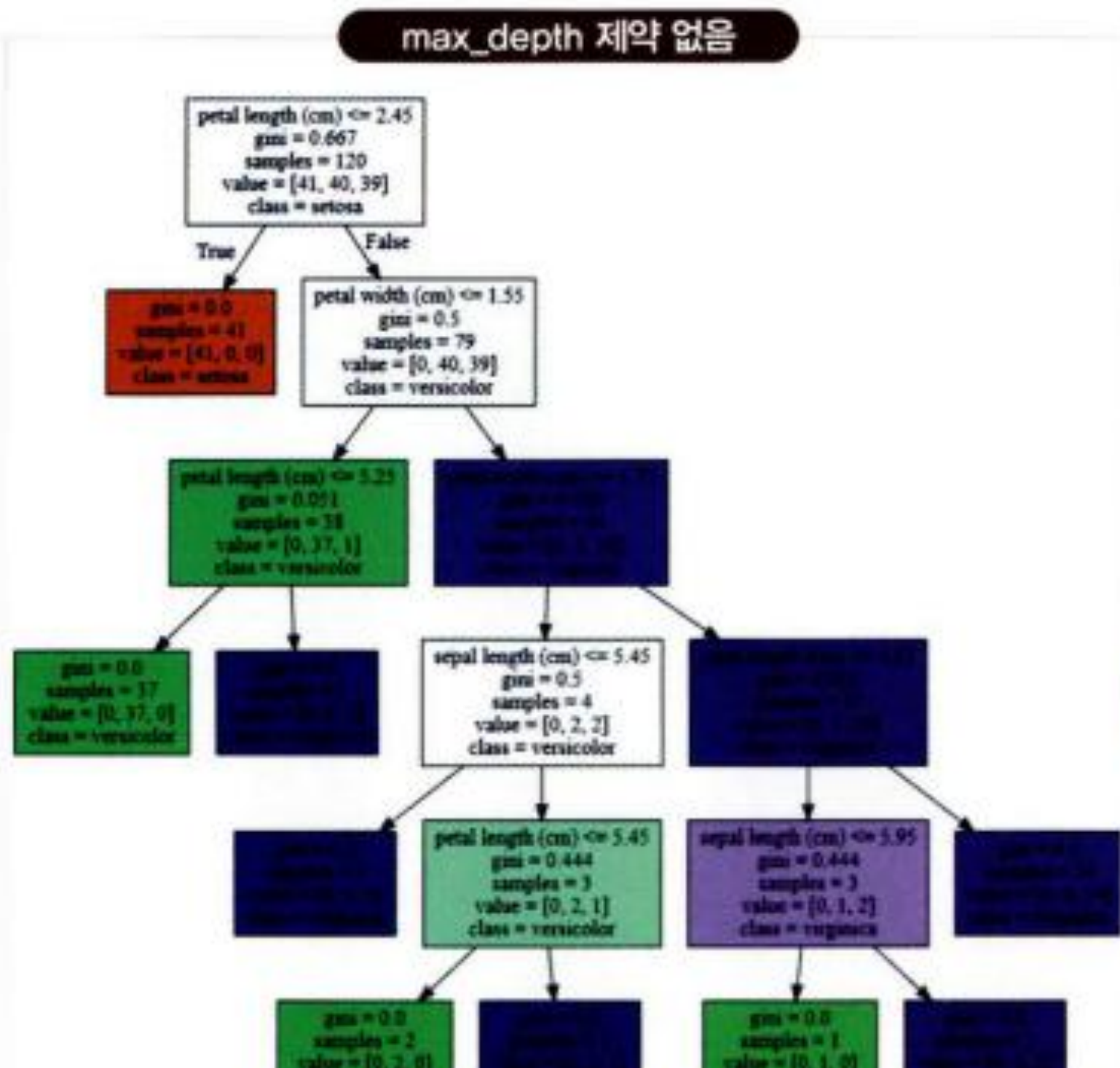
- 지니계수 0.5로 높음
- 데이터 혼재
- 분할 규칙 조건 생성

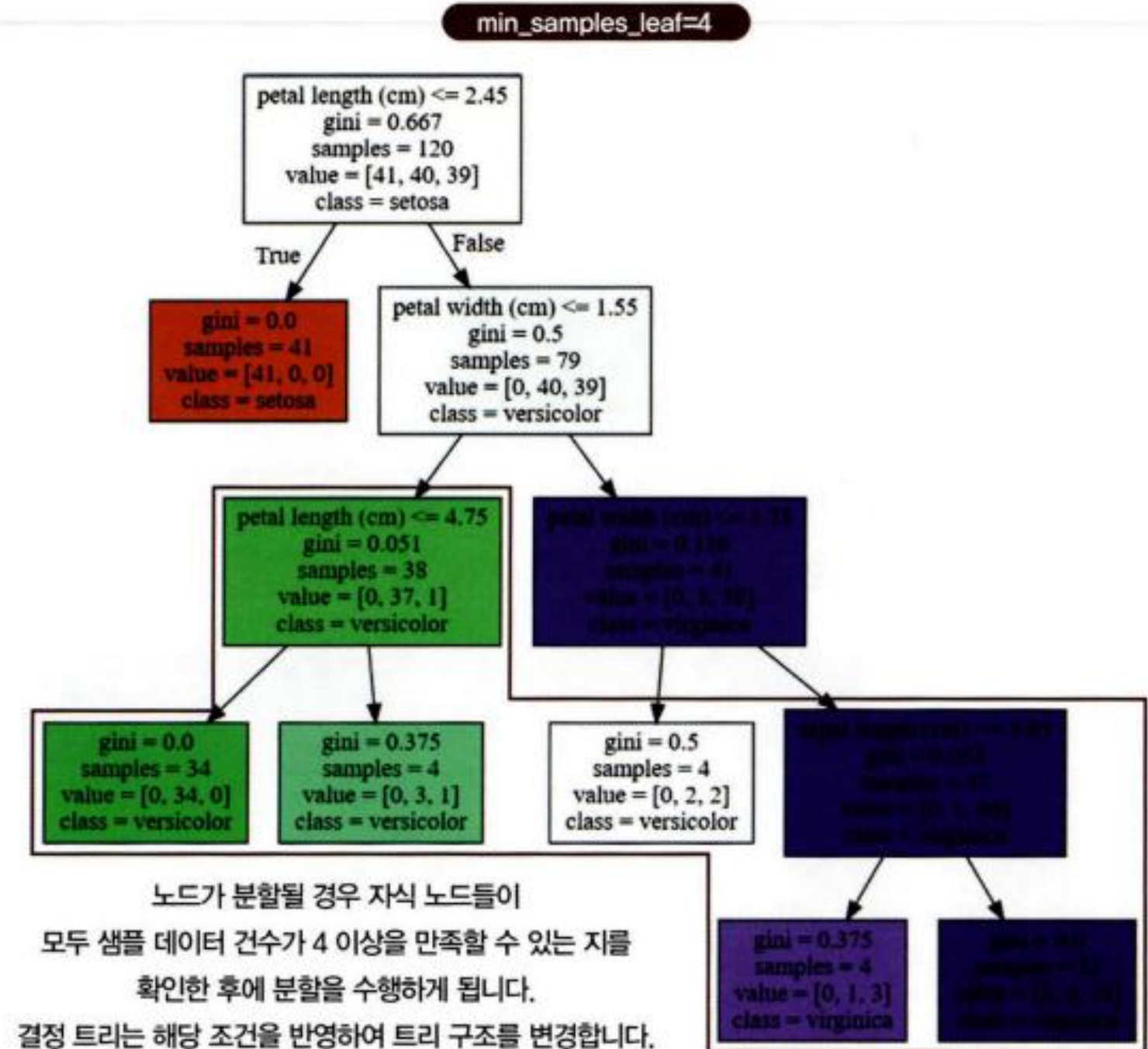
- 지니계수 낮음
- 하지만 데이터가 혼재되어 있어서 분할
- 과적합 문제 발생 가능

결정 트리 알고리즘을 제어하는 대부분 **하이퍼 파라미터** 대부분은 복잡한 트리 생성을 방지하기 위함

1. `max_depth()`: 결정 트리의 최대 트리 깊이
2. `min_samples_splits()`: 분할할 때 필요한 최소한의 샘플 데이터 개수
3. `min_samples_leaf()`: 분할 후 왼쪽과 오른쪽 자식 노드(리프 노드) 각각이 가지게 될 최소 데이터 건수

#3 결정 트리 모델의 시각화



EWCHA
EUROPEAN

노드가 분할될 경우 자식 노드들이
모두 샘플 데이터 건수가 4 이상을 만족할 수 있는 지를
확인한 후에 분할을 수행하게 됩니다.
결정 트리는 해당 조건을 반영하여 트리 구조를 변경합니다.

#3 결정 트리 모델의 시각화

#1 DecisionTreeClassifier 객체의 **feature_importances_** 속성

- 피처의 중요도가 ndarray 형태로 피처 순서대로 값이 할당
- 피처가 트리 분할 시 정보 이득이나 지니 계수를 효율적으로 개선시킨 수준을 정규화된 값으로 표현
- 값이 높을수록 해당 피처의 중요도가 높음

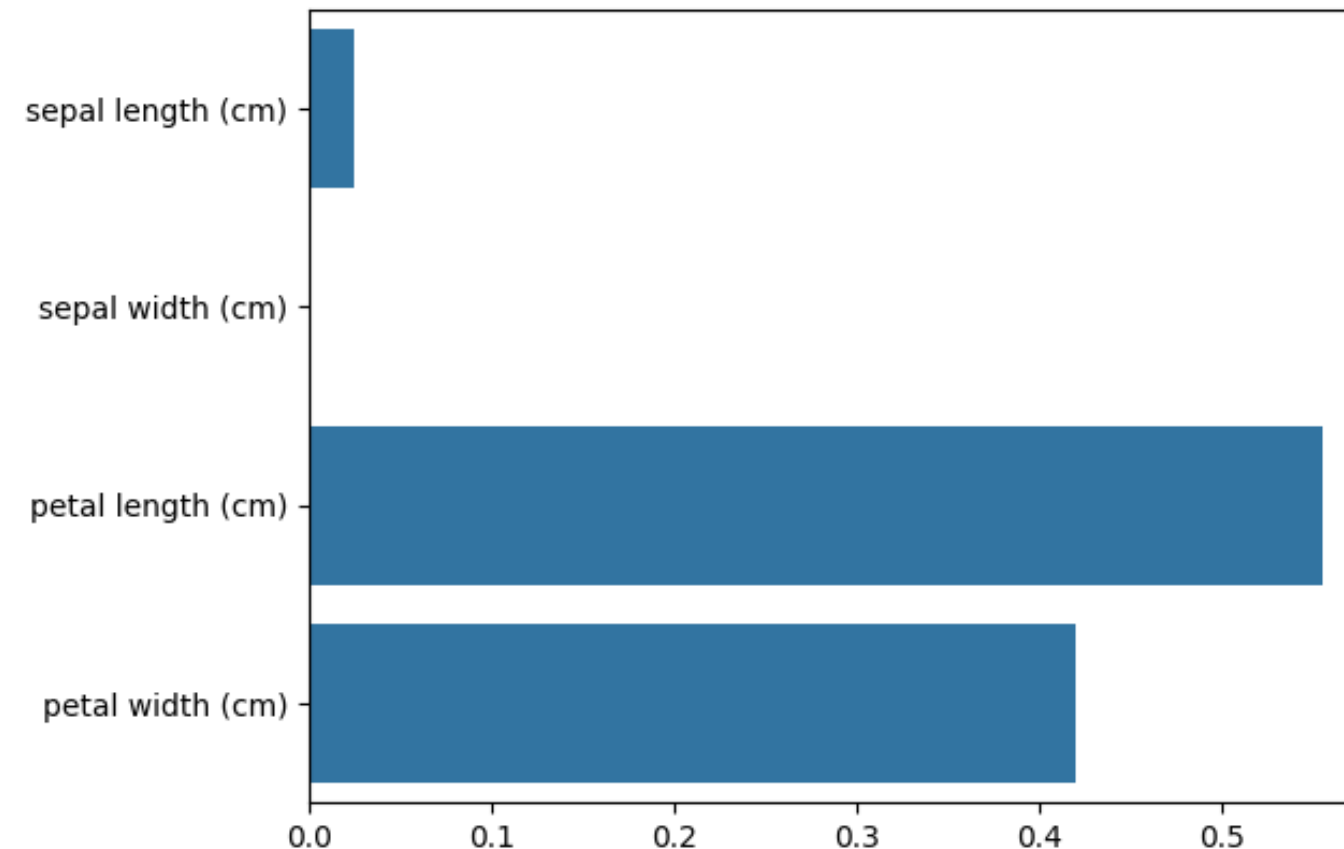
```
import seaborn as sns
import numpy as np
%matplotlib inline

#feature importance 추출
print("Feature
importances:\n{0}".format(np.round(dt_clf.feature_importances_, 3)))

for name, value in zip(iris_data.feature_names,
dt_clf.feature_importances_):
    print("{0} : {1:.3f}".format(name, value))

sns.barplot(x = dt_clf.feature_importances_, y = iris_data.feature_names)
```

```
Feature importances:
[0.025 0.    0.555 0.42 ]
sepal length (cm) : 0.025
sepal width (cm) : 0.000
petal length (cm) : 0.555
petal width (cm) : 0.420
<Axes: >
```



#4 결정 트리 과적합(Overfitting)

#1 make_classification()

분류를 위한 테스트용 데이터를 쉽게 만들 수 있도록

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt
%matplotlib inline
```

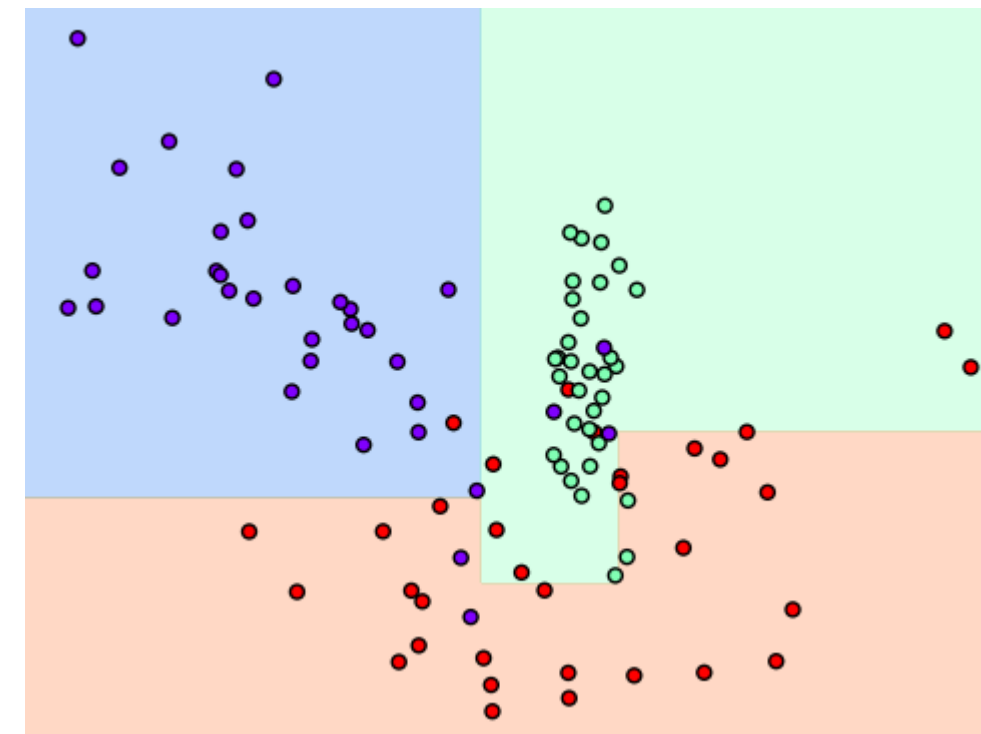
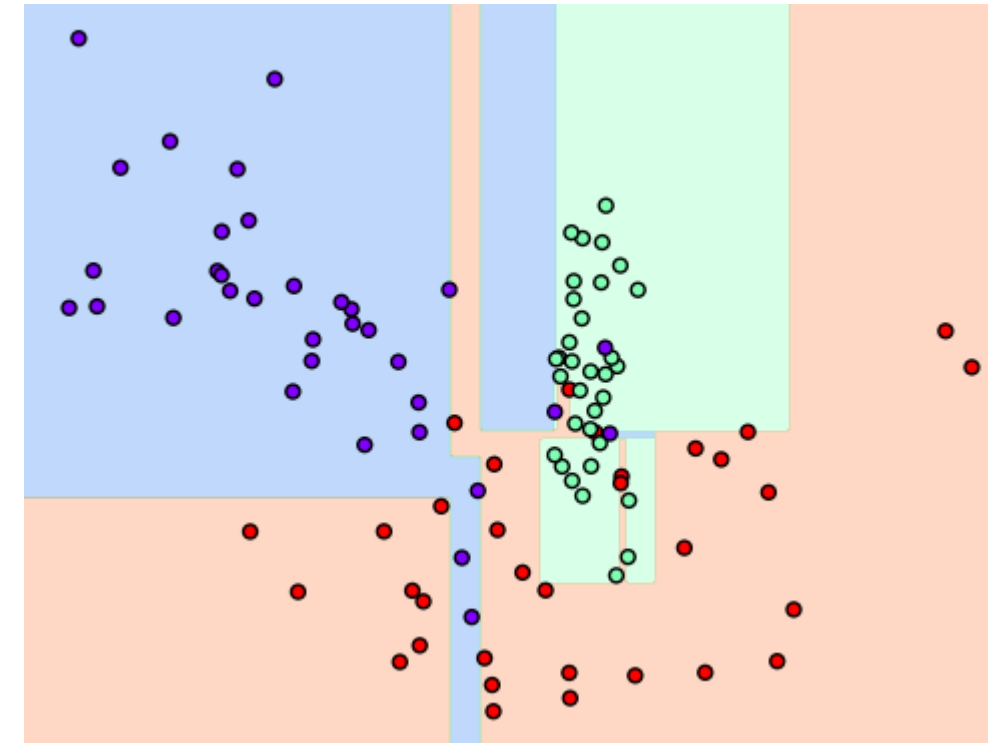
```
plt.title("3 Class values with 2 Features Sample data creation")
```

```
#2차원 시각화를 위해서는 피쳐 2개, 클래스 3가지 유형의 분류 샘플 데이터 생성
X_features, y_labels = make_classification(n_features=2, n_redundant=0, n_informative=2,
                                         n_classes=3, n_clusters_per_class=1, random_state = 0)
```

```
#그래프 형태로 2개의 피쳐로 2차원 좌표 시각화, 각 클래스 값은 다른 색깔로 표시됨
plt.scatter(X_features[:,0], X_features[:,1], marker = 'o', c=y_labels, s=25, edgecolors='k')
```

```
dt_clf = DecisionTreeClassifier(random_state=156).fit(X_features, y_labels)
visualize_boundary(dt_clf, X_features, y_labels)
```

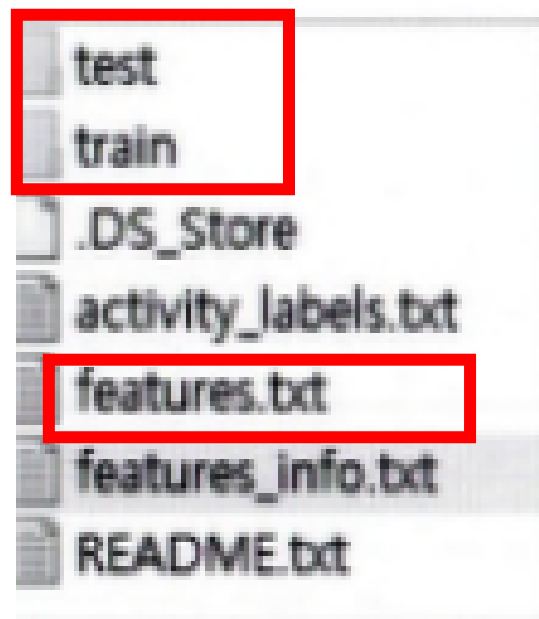
```
dt_clf = DecisionTreeClassifier(min_samples_leaf=6, random_state=156).fit(X_features, y_labels)
```



#5 결정 트리 실습-사용자 행동 인식 데이터 세트

*UCI 머신러닝 리포지토리 (Machine Learning Repository) 제공 사용자 행동 인식(Human Activity Recognition) 데이터 세트
: 30명에게 스마트폰 센서를 장착한 뒤 사람의 동작과 관련된 여러 가지 피처를 수집한 데이터

라벨&피처 데이터 값



피처 이름

#1 데이터 로드 후 중복된 피처명을 피처명_1, 피처명_2로 바꾸기
: 판다스에서 중복된 피처명으로 데이터 파일을 DataFrame에 로드하면 오류 발생

```
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline
```

```
#features.txt 파일에는 피처 이름 index와 피처명이 공백으로 분리되어 있음. 이를
DataFrame으로 로드
feature_name_df = pd.read_csv('UCI HAR Dataset/features.txt', sep = '\s+',
                             header = None, names = ['column_index', 'column_name'])
```

```
#중복된 피처명 개수 세기
feature_dup_df = feature_name_df.groupby('column_name').count()
print(feature_dup_df[feature_dup_df['column_index'] > 1].count())
feature_dup_df[feature_dup_df['column_index'] > 1].head()
```

42개 중복

column_index
dtype: int64

42

column_index	
column_name	
fBodyAcc-bandsEnergy0-1,16	3
fBodyAcc-bandsEnergy0-1,24	3
fBodyAcc-bandsEnergy0-1,8	3
fBodyAcc-bandsEnergy0-17,24	3
fBodyAcc-bandsEnergy0-17,32	3

#5 결정 트리 실습-사용자 행동 인식 데이터 세트

#중복 피처명에 _1 _2 달기

```
def get_new_feature_name_df(old_feature_name_df):
    old_feature_name_df['cumcnt'] =
old_feature_name_df.groupby('column_name').cumcount()

    old_feature_name_df['new_name'] = old_feature_name_df[['column_name',
'cumcnt']].apply(lambda x: x[0] + '_' + str(x[1]) if x[1] > 0 else x[0], axis =1)
    new_feature_name_df = old_feature_name_df.drop('cumcnt', axis =1)
    return new_feature_name_df
```

- 학습/테스트용 피처 및 레이블 데이터 세트 DataFrame에 로드.
- 각 데이터 파일은 공백 분리
- 레이블의 칼럼은 'action'으로 명명

```
import pandas as pd
```

```
def get_human_dataset():
```

```
    feature_name_df = pd.read_csv('UCI HAR Dataset/features.txt', sep = '\s+',
                                header = None, names = ['column_index', 'column_name'])
    new_feature_name_df = get_new_feature_name_df(feature_name_df)
    new_feature_name_list = new_feature_name_df['new_name'].tolist()
```

```
#데이터 로드
```

```
X_train = pd.read_csv('UCI HAR Dataset/train/X_train.txt', sep = '\s+',
                      header = None, names = new_feature_name_list)
X_test = pd.read_csv('UCI HAR Dataset/test/X_test.txt', sep = '\s+',
                     header = None, names = new_feature_name_list)
y_train = pd.read_csv('UCI HAR Dataset/train/y_train.txt', sep = '\s+',
                      header = None, names = ['action'])
y_test = pd.read_csv('UCI HAR Dataset/test/y_test.txt', sep = '\s+',
                     header = None, names = ['action'])
```

```
    return X_train, X_test, y_train, y_test
```

```
X_train, X_test, y_train, y_test = get_human_dataset()
```

#5 결정 트리 실습-사용자 행동 인식 데이터 세트

#2 최적 하이퍼 파라미터 찾기: depth 크기

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import accuracy_score

dt_clf = DecisionTreeClassifier(random_state=134)
dt_clf.fit(X_train, y_train)
pred = dt_clf.predict(X_test)
accuracy = accuracy_score(y_test, pred)
print('정확도: {0:.4f}'.format(accuracy))
print('DecisionTreeClassifier 기본 파라미터:\n', dt_clf.get_params())
```

```
from sklearn.model_selection import GridSearchCV

params = {
    'max_depth': [6, 8, 10, 12, 16, 20, 24],
    'min_samples_split': [16, 24]
}

grid_cv = GridSearchCV(dt_clf, param_grid=params, scoring = 'accuracy', cv = 5, verbose = 1)
grid_cv.fit(X_train, y_train)
print('최고 평균 정확도 수치: {0:.4f}'.format(grid_cv.best_score_))
print('최적 하이퍼 파라미터: ', grid_cv.best_params_)
```

Fitting 5 folds for each of 14 candidates, totalling 70 fits

최고 평균 정확도 수치: 0.8516

최적 하이퍼 파라미터: {'max_depth': 8, 'min_samples_split': 16}

#GridSearchCV 객체의 cv_results_ 속성
: 각 하이퍼파라미터 조합별 교차검증
성능(평균, 표준편차, 각 fold 결과 등)을
딕셔너리 형태로 반환

#GridSearchCV 객체의 cv_results_ 속성을 df로 생성

```
cv_results_df = pd.DataFrame(grid_cv.cv_results_)
cv_results_df[['param_max_depth', 'mean_test_score']]
```



	param_max_depth	mean_test_score
0	6	0.844125
1	8	0.851613
2	10	0.845495
3	12	0.845360
4	16	0.842775
5	20	0.843862
6	24	0.843862

#5 결정 트리 실습-사용자 행동 인식 데이터 세트

```
max_depths = [ 6, 8, 10, 12, 16, 20, 24]
# max_depth 값을 변화시키면서 그때마다 학습과 테스트 세트에서의 예측 성능 측정
for depth in max_depths:
    dt_clf = DecisionTreeClassifier(max_depth = depth, min_samples_split=16, random_state=156)
    dt_clf.fit(X_train, y_train)
    pred = dt_clf.predict(X_test)
    accuracy = accuracy_score(y_test, pred)
    print('max_depth = {0} 정확도: {1:.4f}'.format(depth, accuracy))
```

```
max_depth = 6 정확도: 0.8551
max_depth = 8 정확도: 0.8717
max_depth = 10 정확도: 0.8599
max_depth = 12 정확도: 0.8571
max_depth = 16 정확도: 0.8599
max_depth = 20 정확도: 0.8565
max_depth = 24 정확도: 0.8565
```

- 마찬가지로 For 문을 활용해 max_depth의 값을 하나 씩 변경시키면서 예측 성능 측정하면 max_depth가 8일 때 가장 높은 정확도가 나타나며, 8을 넘어가면 정확도가 감소한다는 것을 알 수 있음

```
best_df_clf = grid_cv.best_estimator_
pred1 = best_df_clf.predict(X_test)
accuracy = accuracy_score(y_test, pred1)
print('결정 트리 예측 정확도: {0:.4f}'.format(accuracy))
```

결정 트리 예측 정확도:0.8717

- GridSearchCV 객체의 속성인 **best_estimator_**는 최적 하이퍼 파라미터인 max depth 8, min_samples_split 16으로 학습이 완료된 Estimator 객체

#5 결정 트리 실습-사용자 행동 인식 데이터 세트

#3 각 피처의 중요도를 시본 막대그래프로 표현

```
import seaborn as sns
```

```
ftr_importances_values = best_df_clf.feature_importances_
```

```
#Top. 중요도로 정렬 쉽게 하고, 시본 막대 그래프로 쉽게 표현하기 위해 Series 변환  
ftr_importances = pd.Series(ftr_importances_values, index=X_train.columns)
```

```
#중요도값 순으로 Series 정렬
```

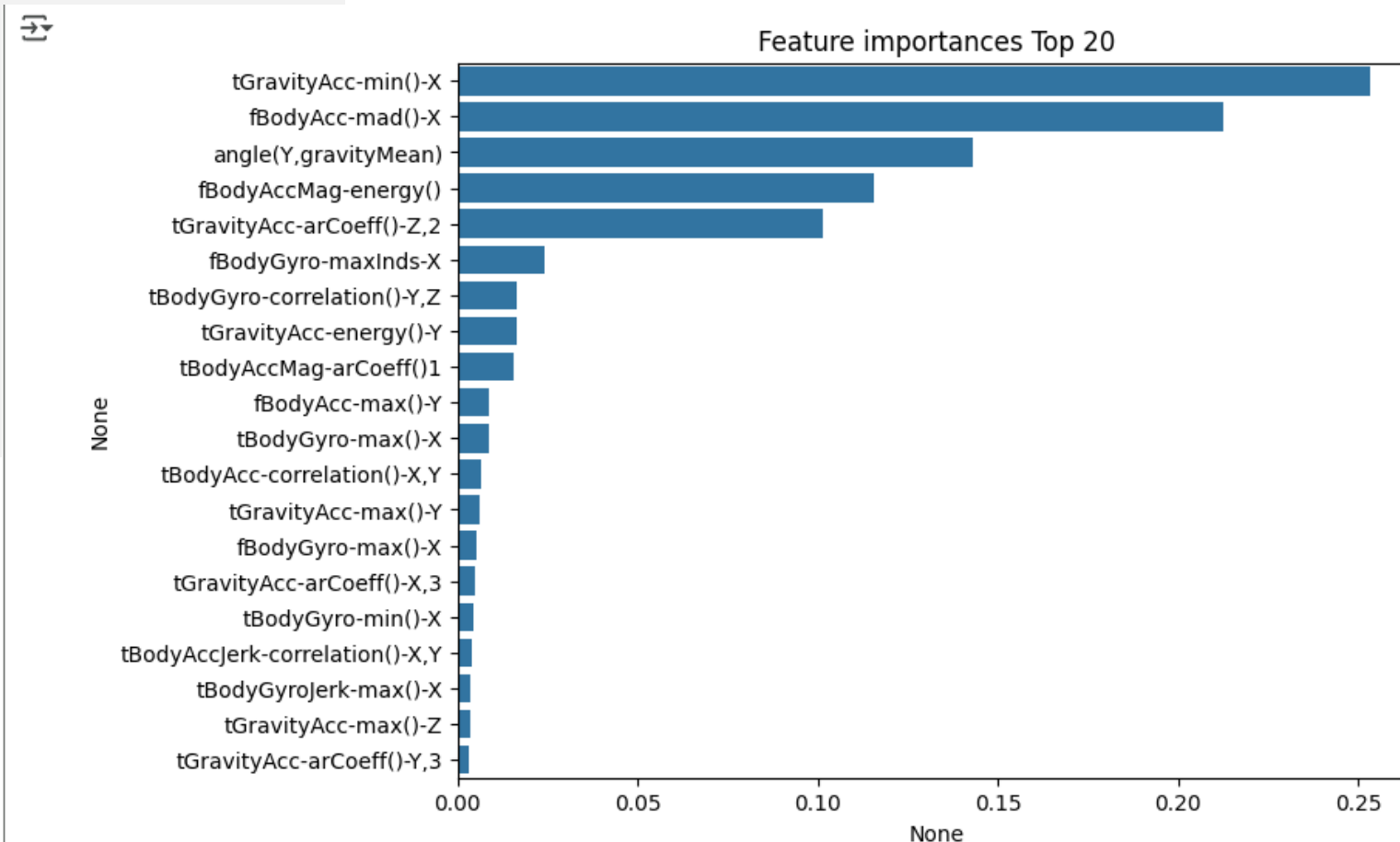
```
ftr_top20 = ftr_importances.sort_values(ascending = False)[:20]
```

```
plt.figure(figsize=(8,6))
```

```
plt.title('Feature importances Top 20')
```

```
sns.barplot(x=ftr_top20, y=ftr_top20.index)
```

```
plt.show()
```



4.3 앙상블 학습



#1 앙상블 학습 개요

앙상블 학습 Ensemble Learning

: 여러 개의 분류기(Classifier)를 생성하고 그 예측을 결합하여 단일 모델보다 **더 정확하고**
안정적인 결과, 신뢰성이 높은 예측값을 도출하는 기법

특징

- **정형 데이터(Structured Data)** 분류/회귀에서 매우 강력한 성능
 - * 이미지/영상/음성과 같은 비정형 데이터는 **딥러닝**이 강세
- 쉽고 직관적인 사용 + 강력한 예측 성능
 - 데이터 분석가와 캐글(Kaggle) 대회 참가자들 사이에서 필수적인 알고리즘

#1 앙상블 학습 개요

대표 알고리즘

- **랜덤 포레스트 (Random Forest)**: 대표적인 배깅(bagging) 알고리즘
- **그래디언트 부스팅 (Gradient Boosting)**: 성능 우수, 하지만 상대적으로 느림
- **XGBoost**: 캐글에서 인기, 강력한 성능
- **LightGBM**: XGBoost보다 빠른 수행 속도 + 유사 성능
- **스태킹 (Stacking)**: 여러 모델의 예측을 기반으로 메타 모델을 학습

#1 앙상블 학습 개요

주요 앙상블 기법

1. 보팅 (Voting)

- 서로 다른 알고리즘의 분류기를 결합 (주로 결정트리)
- 다수결(majority voting)로 최종 결정

2. 배깅 (Bagging)

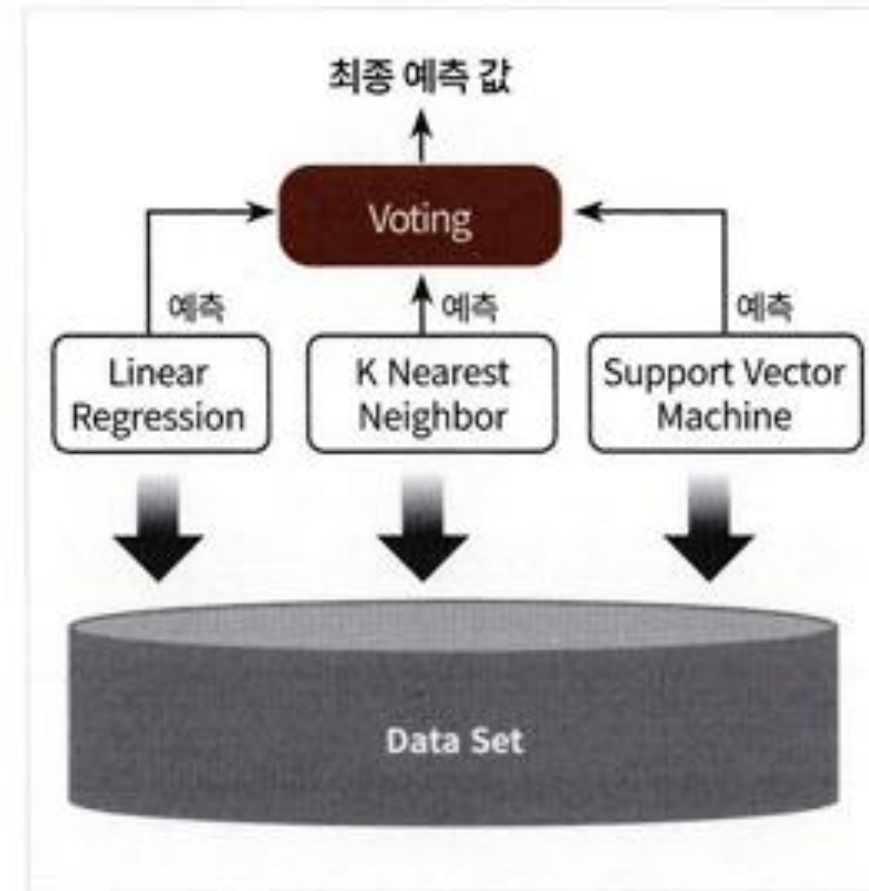
- 같은 알고리즘 기반, 샘플링을 달리하여 학습
- 부트스트래핑(Bootstrapping) 분할 방식 : 개별 Classifier에게 데이터를 샘플링 해서 추출
- Ex) 랜덤 포레스트

3. 부스팅 (Boosting)

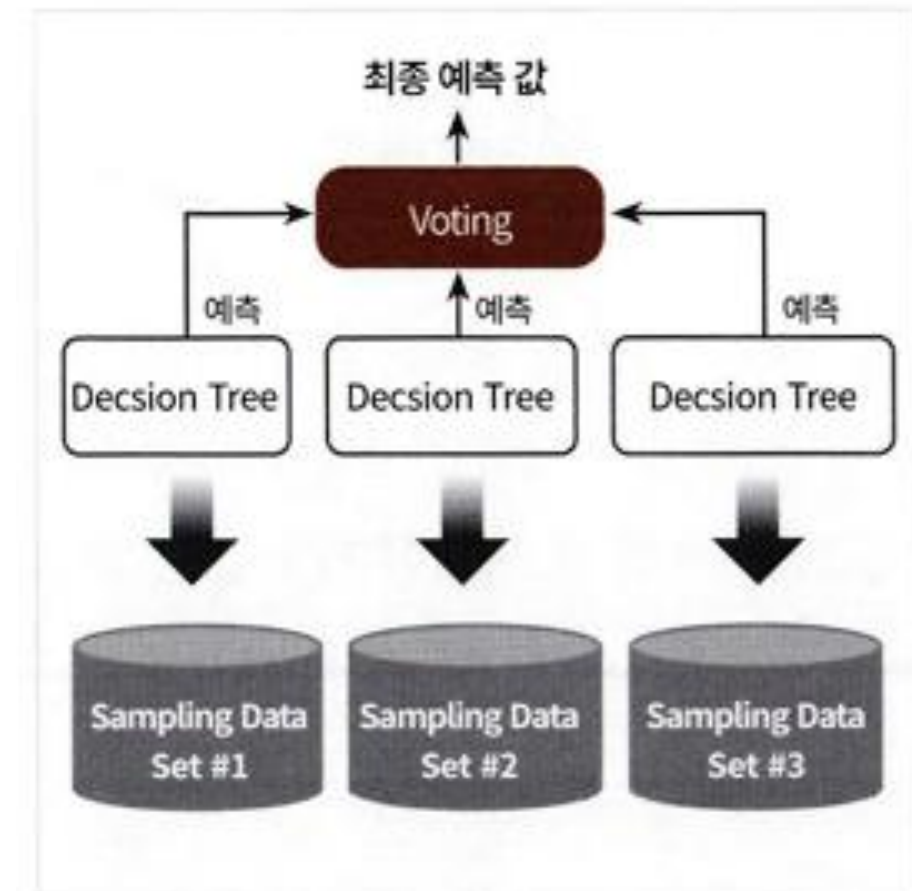
- 이전 모델의 오차를 보완하는 방식으로 순차적 학습
- 이전 모델이 틀린 데이터에 가중치(weight)를 부여 → 다음 모델이 보완 학습
- Ex) Gradient Boosting, XGBoost, LightGBM

4. 스택킹 (Stacking)

- 여러 모델의 결과를 입력값으로 받아 메타 모델이 최종 예측



Voting 방식

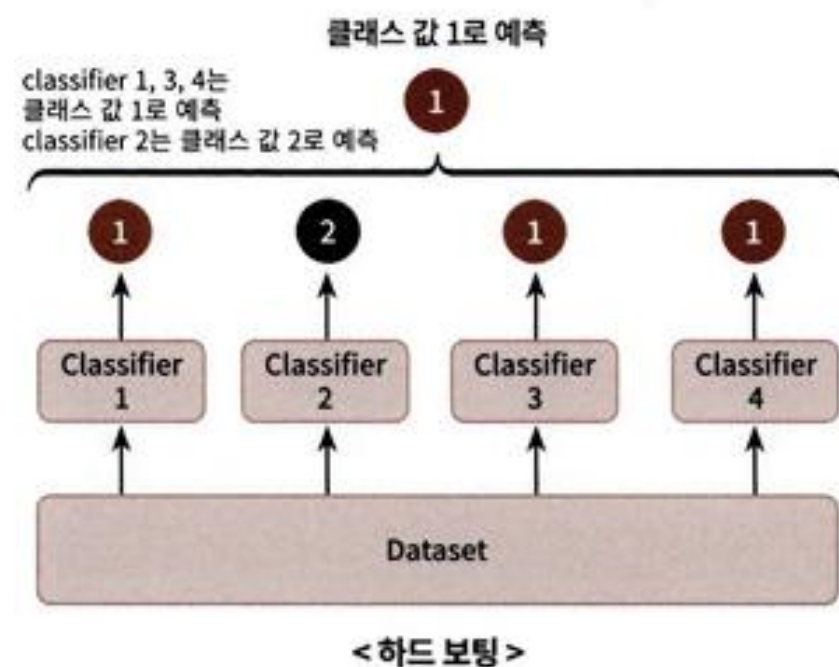


Bagging 방식

#2 보팅 유형 - 하드 보팅 & 소프트 보팅

Hard Voting 하드 보팅

- 다수결 원칙 기반
- 각 분류기의 최종 예측 (class label)을 모아
→ 가장 많이 선택된 클래스를 최종 결과로 결정

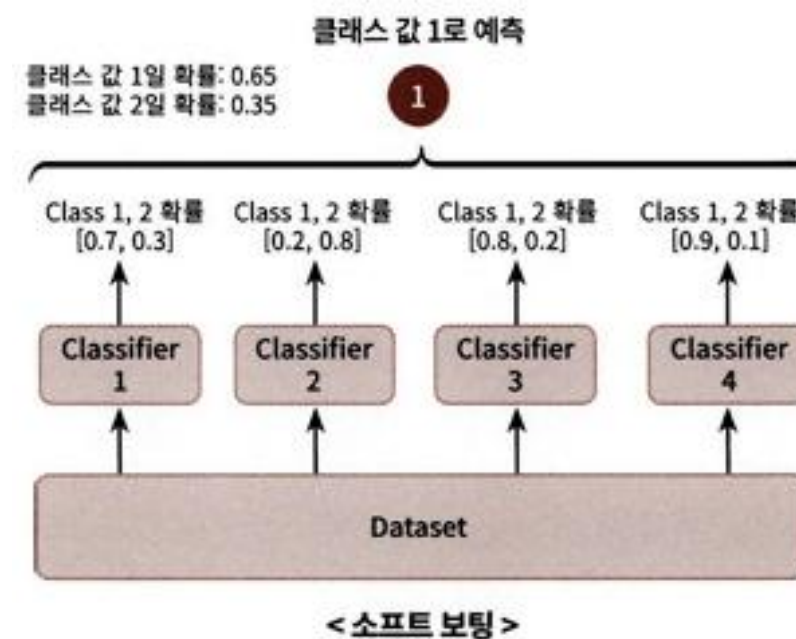


- Classifier1 → Class 1
- Classifier2 → Class 2
- Classifier3 → Class 1
- Classifier4 → Class 1

👉 최종 결과: Class 1 (다수결)

Soft Voting 소프트 보팅

- 예측 확률 (Probability)을 평균하여 최종 결과 결정
- 예측 성능이 더 좋아, 하드 보팅보다 자주 사용됨
- 각 분류기가 출력한 클래스별 확률을 합산
→ 평균 → 가장 높은 확률 클래스를 최종 결과로 결정



- Class 1 확률 평균 = $(0.7 + 0.2 + 0.8 + 0.9) / 4 = 0.65$
- Class 2 확률 평균 = $(0.3 + 0.8 + 0.2 + 0.1) / 4 = 0.35$

👉 최종 결과: Class 1 (확률 0.65)

#3 보팅 분류기 (voting classifier) 실습

Data Set

- 위스콘신 유방암 데이터 (breast cancer dataset)
- 종양의 크기·모양 등 다양한 특성(feature) 포함
- 목표: 악성(Malignant) vs 양성(Benign) 여부 예측 (이진 분류)

ML Model

- 기반 분류기: 로지스틱 회귀(Logistic Regression) & KNN(K-Nearest Neighbors)
- 보팅 방식: 소프트 보팅 (Soft Voting)

```
import pandas as pd

from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1 데이터 로드
cancer = load_breast_cancer()
data_df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
data_df.head(3)
```

#3 보팅 분류기 (voting classifier) 실습

개별 모델 - 로지스틱 회귀, KNN

```
lr_clf = LogisticRegression(solver='liblinear')
knn_clf = KNeighborsClassifier(n_neighbors=8)
```

소프트 보팅 기반 앙상블 모델

voting 기본 'hard' -> 'soft'

```
vo_clf = VotingClassifier(estimators=[('LR', lr_clf), ('KNN', knn_clf)], voting='soft')
```

② 데이터 분할

```
X_train, X_test, y_train, y_test = train_test_split(
    cancer.data, cancer.target, test_size=0.2, random_state=156
)
```

③ VotingClassifier 학습/예측/평가

```
vo_clf.fit(X_train, y_train)
pred = vo_clf.predict(X_test)
print('Voting 분류기 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

④ 개별 모델 학습/예측/평가

```
classifiers = [lr_clf, knn_clf]
for classifier in classifiers:
    classifier.fit(X_train, y_train)
    pred = classifier.predict(X_test)
    class_name = classifier.__class__.__name__
    print('{0} 정확도: {1:.4f}'.format(class_name, accuracy_score(y_test, pred)))
```

1. 데이터 준비

- 학습/테스트 데이터를 8:2 비율로 분할

2. 개별 분류기 정의

- ① 로지스틱 회귀 (Logistic Regression)
- ② K-최근접 이웃 (KNN, n_neighbors=8)

3. 앙상블 모델 생성

- VotingClassifier를 이용해 두 분류기를 결합
- voting='soft' → 각 분류기의 **예측 확률**을 평균내 최종 클래스 결정

4. 모델 학습 및 평가

- 앙상블 모델(vo_clf)과 개별 모델을 모두 학습
- accuracy_score로 정확도 비교

5. 결과 해석

- 보팅 분류기는 여러 개의 분류기를 결합해 **유연성과 안정성을 높일 수 있음**
- 하지만 항상 개별 모델보다 성능이 높아지는 것은 아니며, 데이터 특성에 따라 다름

```
Voting 분류기 정확도: 0.9561
LogisticRegression 정확도: 0.9474
KNeighborsClassifier 정확도: 0.9386
```

#4 배깅 (bagging)

- 같은 알고리즘으로 여러 분류기를 학습시키는 방법
- 훈련 데이터 샘플링 방식에 따라 구분:

배깅 (Bagging, Bootstrap Aggregating)

- 훈련 샘플을 **중복 허용**하여 무작위 추출
→ 개별 분류기 학습
- 다양성 ↑ → 앙상블 분산 ↓
- 대표 알고리즘: **랜덤 포레스트**

페이스팅 (Pasting)

- 훈련 샘플을 **중복 없이** 무작위 추출
→ 개별 분류기 학습
- 배깅보다 편향 낮음, 분산 감소 효과는 약간 떨어짐

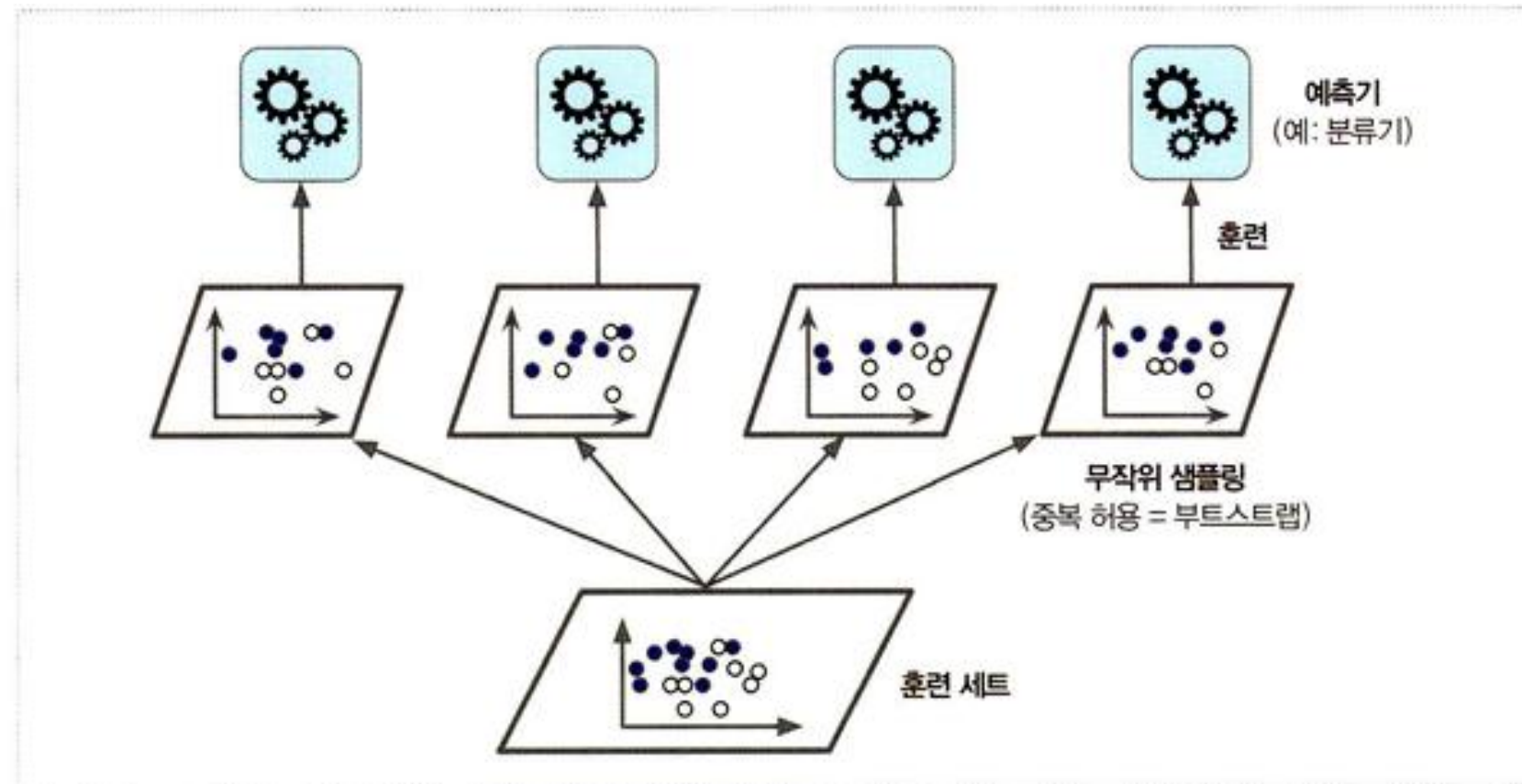


그림 7-4 배깅과 페이스팅은 훈련 세트에서 무작위로 샘플링하여 여러 개의 예측기를 훈련합니다.

#4 배깅 (bagging)

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

① 데이터 로드

```
cancer = load_breast_cancer()
X, y = cancer.data, cancer.target
```

② 데이터 분할 (훈련 80%, 테스트 20%)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

③ BaggingClassifier 정의 (기본 모델: 결정 트리)

```
bag_clf = BaggingClassifier(
    DecisionTreeClassifier(),
    n_estimators=500,    # 트리 개수
    max_samples=100,    # 각 트리 학습용 샘플 수
    bootstrap=True,      # True=배깅, False=페이스팅
    n_jobs=-1           # 모든 CPU 코어 사용
)
```

④ 모델 학습, 예측, 평가 (정확도)

```
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
```

```
print('Bagging 분류기 정확도: {0:.4f}'.format(accuracy_score(y_test, y_pred)))
```

1. 데이터 준비

- 사이킷런의 위스콘신유방암 데이터셋
- 학습/테스트 데이터를 8:2 비율로 분할

2. BaggingClassifier 정의

- 기본 모델: 결정 트리(Decision Tree)
- Bootstrap = True 중복 샘플 허용 (배깅 수행)

3. 모델 학습 및 평가

- fit() BaggingClassifier 학습
- predict() 테스트 데이터에 대한 예측
- 여러 개의 예측기 학습
- 개별 예측기의 결과를투표(분류) / 평균(회귀)하여 최종 예측 생성

4. 결과 해석

- 개별 결정 트리보다 분산 감소 → 일반화 성능 향상
- 개별 모델의 편향은 약간 있지만, 앙상블을 통해 분산 감소
- 단일 결정 트리보다 안정적이고 일반화 성능 우수병렬 학습 가능 → 대규모 앙상블에도 효율적

Bagging 분류기 정확도: 0.9561

#4 배깅 (bagging)

- 개별 예측기는 편향이 조금 높지만, 앙상블을 통해 분산 감소 → 일반화 성능 향상

결정 경계 (Decision Boundary)

- 분류 모델이 클래스를 구분하는 기준선/면
- 단일 결정 트리는 데이터에 따라 복잡하고 불규칙한 경계를 만들 수 있음
= 과적합 위험

비슷한 편향, 더 작은 분산

- 편향(Bias): 모델이 문제를 단순화하면서 생기는 오류
- 분산(Variance): 모델이 훈련 데이터의 작은 변동에도 민감하게 반응하는 정도 (트리가 복잡해 과적합)
- 배깅은 개별 트리의 편향 수준은 그대로 두지만, 여러 트리의 결과를 합쳐 평균내 분산을 줄임
=> 안정적인 예측

일반화 (Generalization)

- 훈련 데이터에만 딱 맞는 모델이 아닌, 새로운 데이터에서도 잘 맞는 모델
- 배깅은 여러 모델의 예측값을 투표/평균하기 때문에 개별 트리의 과적합 감소
= 새로운 데이터에도 성능이 안정적

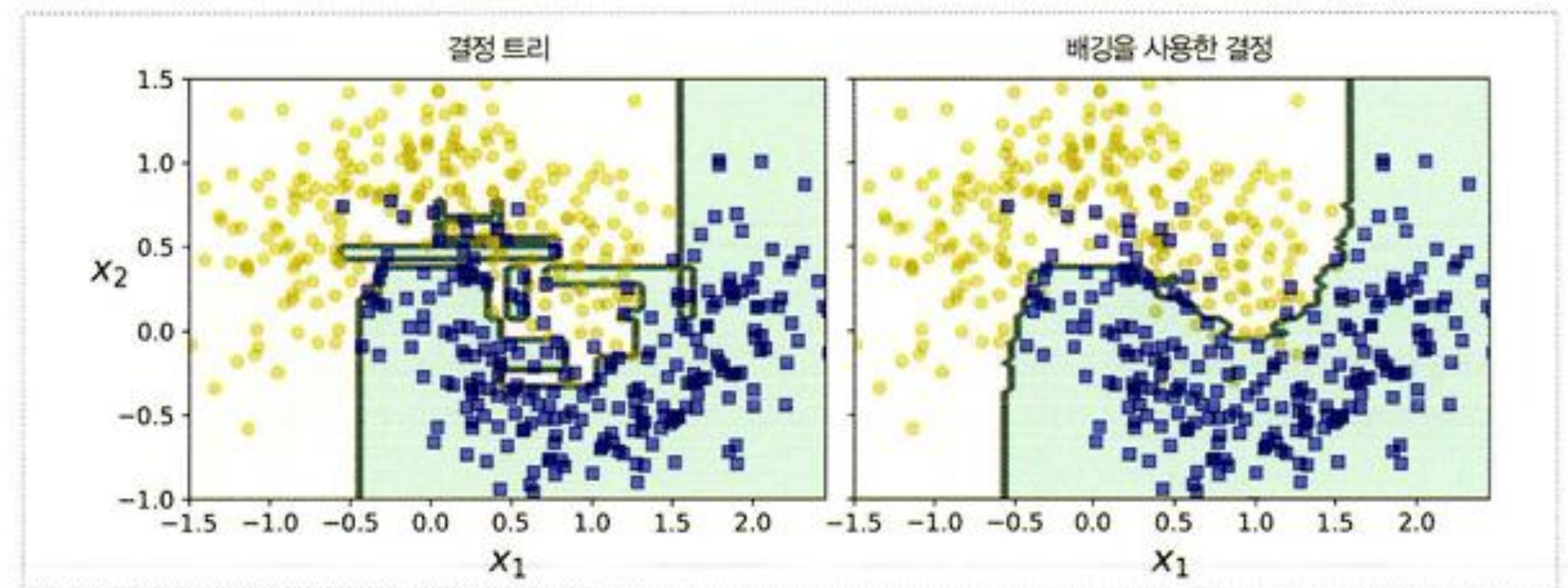


그림 7-5 단일 결정 트리(왼쪽)와 500개 트리로 만든 배깅 앙상블(오른쪽) 비교

#4 배깅 (bagging)

특성 샘플링

- **BaggingClassifier**는 단순히 훈련 샘플만 랜덤하게 뽑아 쓰는 게 아니라, **입력 특성(feature)**도 무작위로 뽑아 사용할 수 있음

매개변수

- `max_features`: 사용할 특성 수
- `Bootstrap_features`: 특성을 중복 허용여부

랜덤 패치 방식 (random patches method)

- 훈련 샘플 + 특성 둘 다 랜덤 샘플링
= 일부 샘플 & 일부 특성 뽑아 학습 진행
- 데이터가 엄청 크거나 이미지처럼 특성이 많은 경우

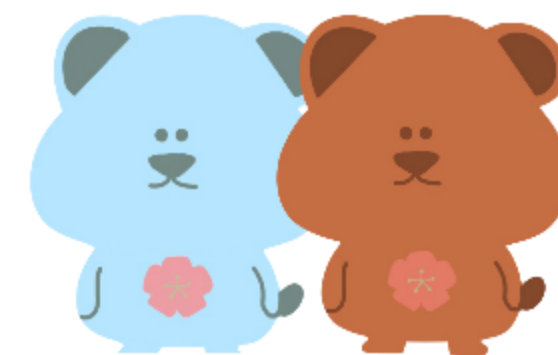
랜덤 서브스페이스 방식 (random subspaces method)

- 샘플은 전부 사용 (`bootstrap=False`, `max_samples=1.0`)
- 대신 특성만 랜덤 샘플링
(`bootstrap_features=True`) or (`max_features < 1.0`)
- 고차원 데이터에서 과적합 방지 + 다양성 증가

효과

- 더 다양한 예측기 생성 → 서로 다른 시각으로 문제를 학습
- 편향(bias)은 조금 증가할 수 있음 (특성을 제한해서 단순화되니까)
- 대신 분산(variance)은 크게 감소 → 과적합 줄이고 일반화 성능 향상

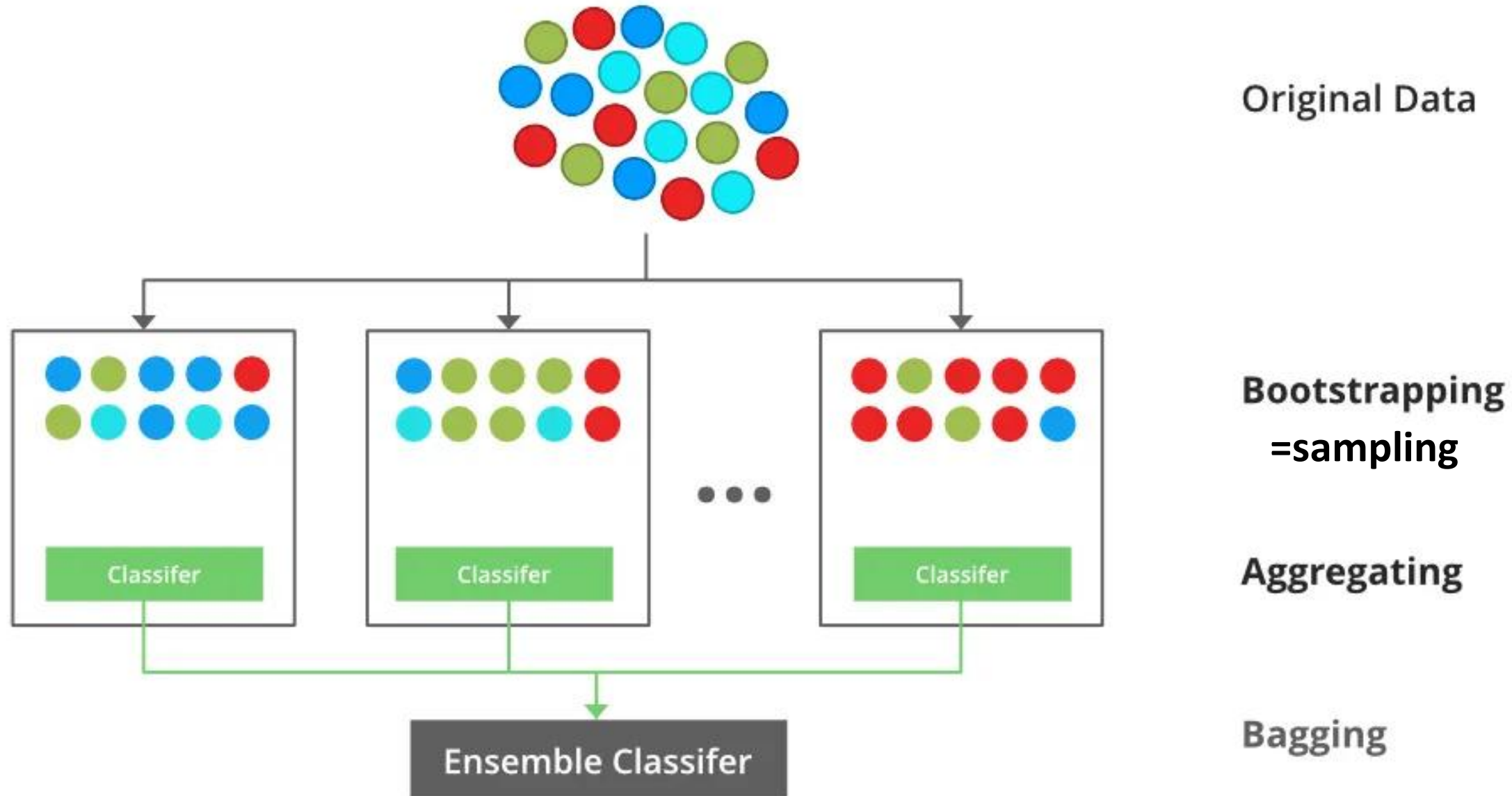
4.4 랜덤 포레스트



#4.1 랜덤 포레스트와 개요 및 실습

#1. Bagging(Bootstrap Aggregating)

: Original Data로부터 data를 랜덤하게 추출해 동일한 크기의 데이터셋을 여러 개 만들고, 각 데이터셋으로 별도의 모델을 학습시킨 후, 이 모델들의 예측 결과를 종합하는 방식



#4.1 랜덤 포레스트와 개요 및 실습

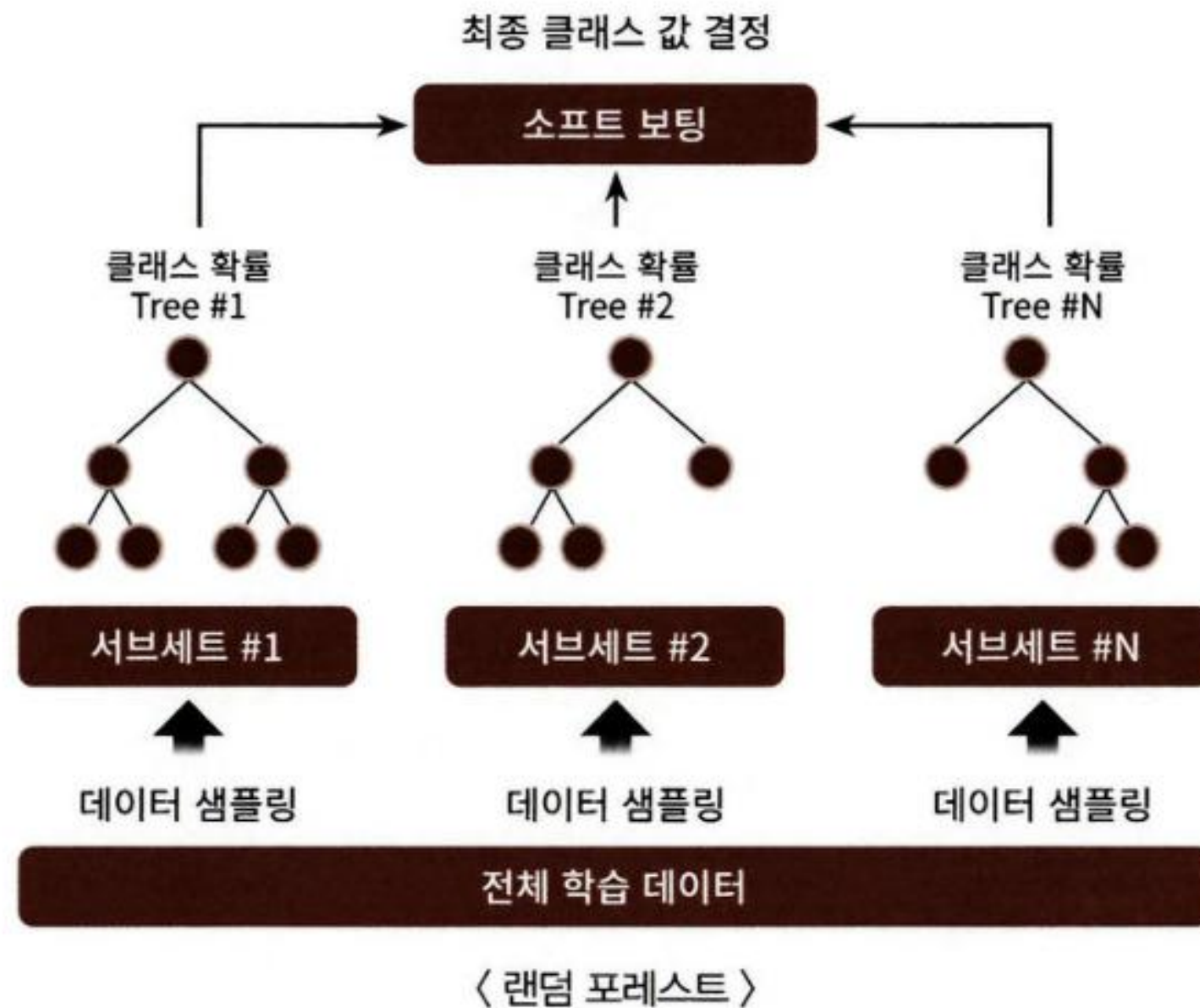
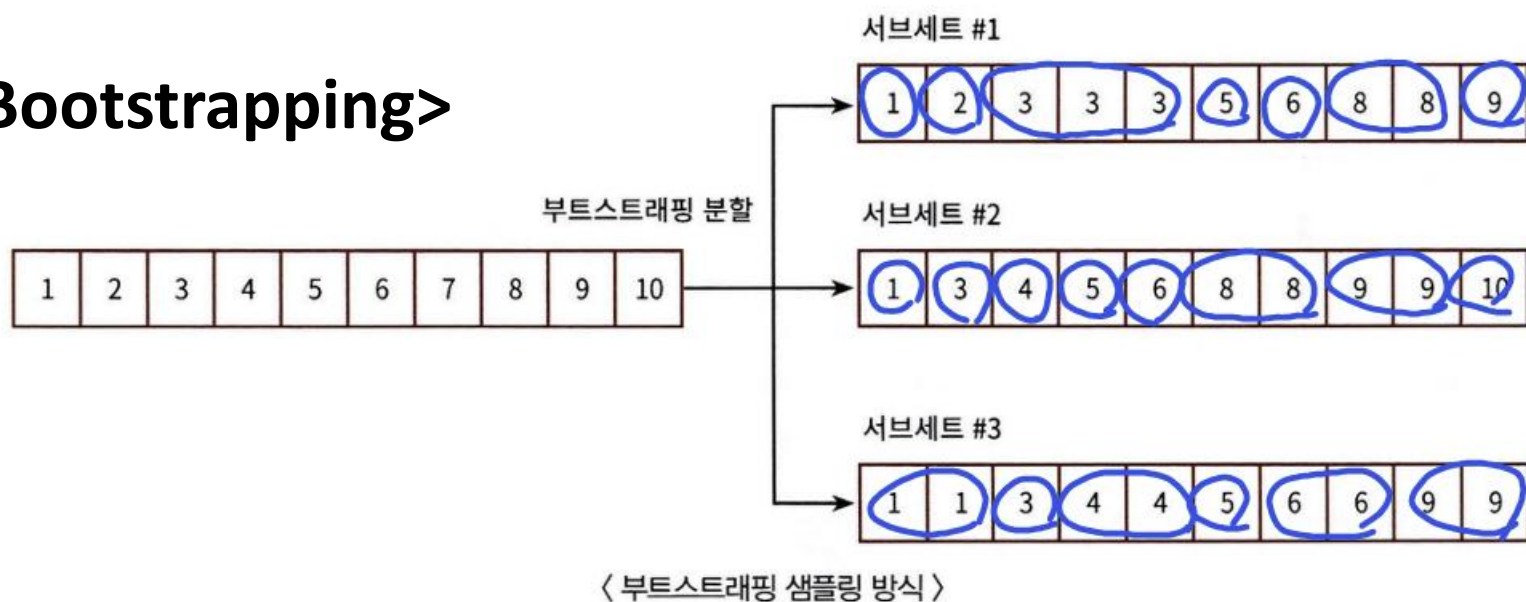
#2. Random Forest

: Bagging의 대표적인 알고리즘
: 여러 개의 결정 트리 분류기가 전체 데이터에서
Bagging 방식으로 각자의 데이터를 샘플링해
개별적으로 학습을 수행한 뒤 보팅을 통해 예측 결정

특징

- 1) 앙상블 알고리즘 중 비교적 빠른 수행 속도
- 2) 높은 예측 성능
- 3) 결정 트리 기반 알고리즘
- 4) 병렬처리

<Bootstrapping>



#4.1 랜덤 포레스트와 개요 및 실습

#3. RandomForestClassifier

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

X_train, X_test, y_train, y_test = get_human_dataset()

rf_clf = RandomForestClassifier(random_state =0, max_depth =8)
rf_clf.fit(X_train, y_train)
pred = rf_clf.predict(X_test)
accuracy = accuracy_score(y_test, pred)
print('랜덤 포레스트 정확도:{0:.4f}'.format(accuracy))
```

랜덤 포레스트 정확도:0.9220

#4.2 랜덤 포레스트 하이퍼 파라미터 및 튜닝

#1. 랜덤 포레스트 하이퍼 파라미터

트리 기반 앙상블 알고리즘의 단점

:하이퍼 파라미터 多 -> 튜닝 시간이 많이 소모됨

=>결정트리에서 사용되는 하이퍼 파라미터와 같은 파라미터가 많은 랜덤포레스트가 하이퍼 파라미터가 적은편!

하이퍼 파라미터 종류

- **n_estimator**: 결정 트리의 개수 지정. Default = 10
- **max_features**: 결정 트리에 사용된 max_feature파라미터와 동일. 기본형:
 'auto'(=='sqrt(제공근)')
=>트리분할 피쳐 참조 시, 전체 피쳐 $\times$, sqrt(전체 피쳐개수)만큼 참조
- **max_depth, min_samples_leaf, min_samples** <=결정 트리에서 과적합 개선하는 파라미터가 똑같이 적용될 수 0

#4.2 랜덤 포레스트 하이퍼 파라미터 및 튜닝

#1.GridSearchCV 이용해 랜덤 포레스트 하이퍼 파라미터 튜닝

[11]

✓ 4분

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
import warnings
warnings.filterwarnings('ignore')

params = {
    'max_depth': [8,16,24],
    'min_samples_leaf': [1,6,12],
    'min_samples_split': [2,8,16]
}
#RandomForestClassifier 객체 생성 후 GridSearchCV 수행
rf_clf = RandomForestClassifier(n_estimators = 100, random_state = 0, n_jobs = -1)
grid_cv = GridSearchCV(rf_clf, param_grid=params, cv=2, n_jobs = -1)
grid_cv.fit(X_train, y_train)

print('최적 하이퍼 파라미터:\n', grid_cv.best_params_)
print('최고 예측 정확도:{0:.4f}'.format(grid_cv.best_score_))
```



최적 하이퍼 파라미터:

{'max_depth': 16, 'min_samples_leaf': 6, 'min_samples_split': 16}

최고 예측 정확도:0.9157

랜덤 포레스트 하이퍼 파라미터 및 튜닝

#1.GridSearchCV 이용해 랜덤 포레스트 하이퍼 파라미터 튜닝

[17]
12
초

```
rf_clf1 = RandomForestClassifier(n_estimators = 100, min_samples_leaf = 6, max_depth=16, min_samples_split =2, random_state = 0)
rf_clf1.fit(X_train, y_train)
pred = rf_clf1.predict(X_test)
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

최적 하이퍼 파라미터로 재학습

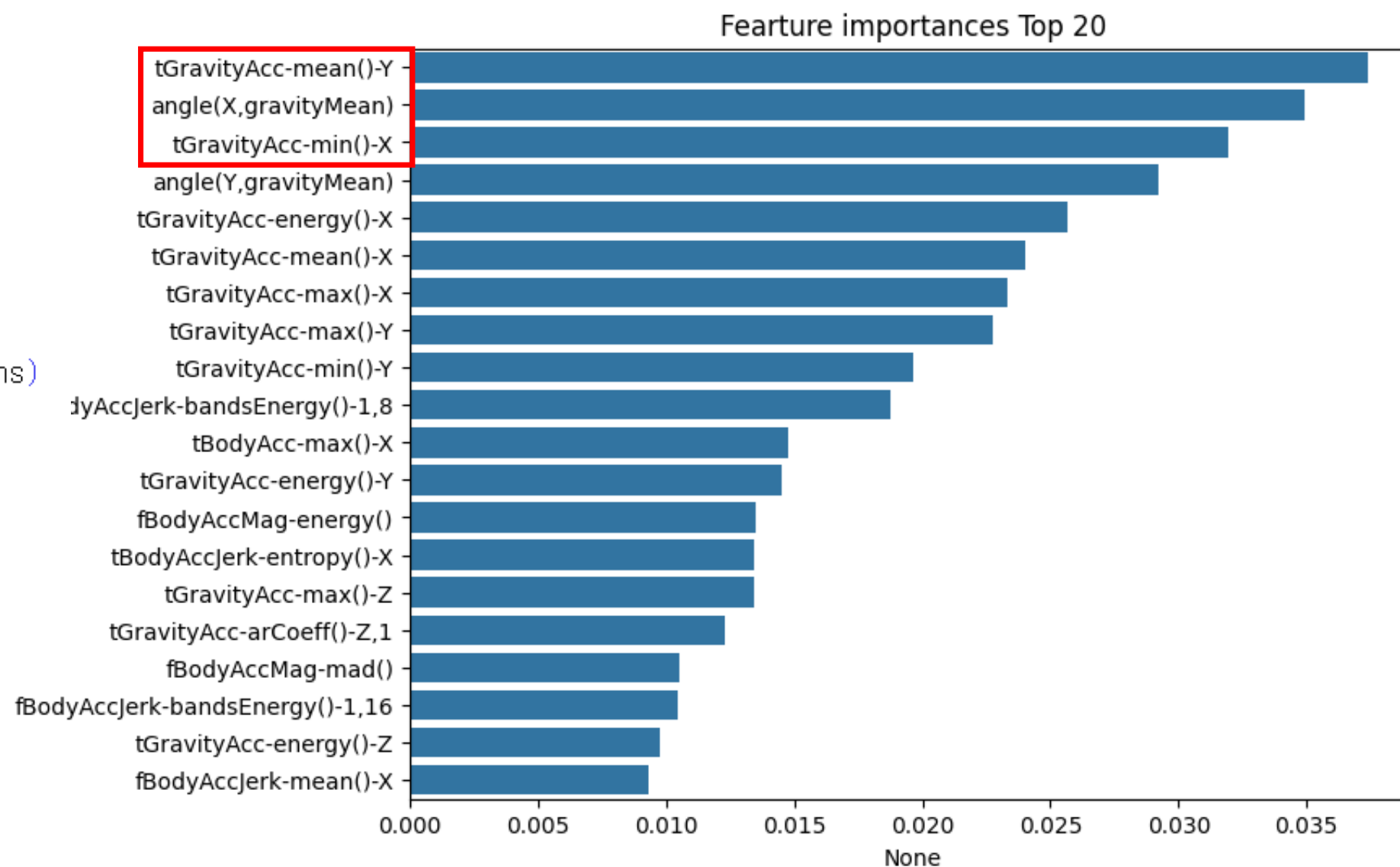
↔ 예측 정확도: 0.9253

<시각화>

```
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
ftr_importances_values = rf_clf1.feature_importances_
ftr_importances = pd.Series(ftr_importances_values, index = X_train.columns)
ftr_top20 = ftr_importances.sort_values(ascending=False)[:20]
```

```
plt.figure(figsize=(8,6))
plt.title('Feature importances Top 20')
sns.barplot(x=ftr_top20, y=ftr_top20.index)
plt.show()
```



+추가자료

5) SVM



#5.1 선형 SVM 분류

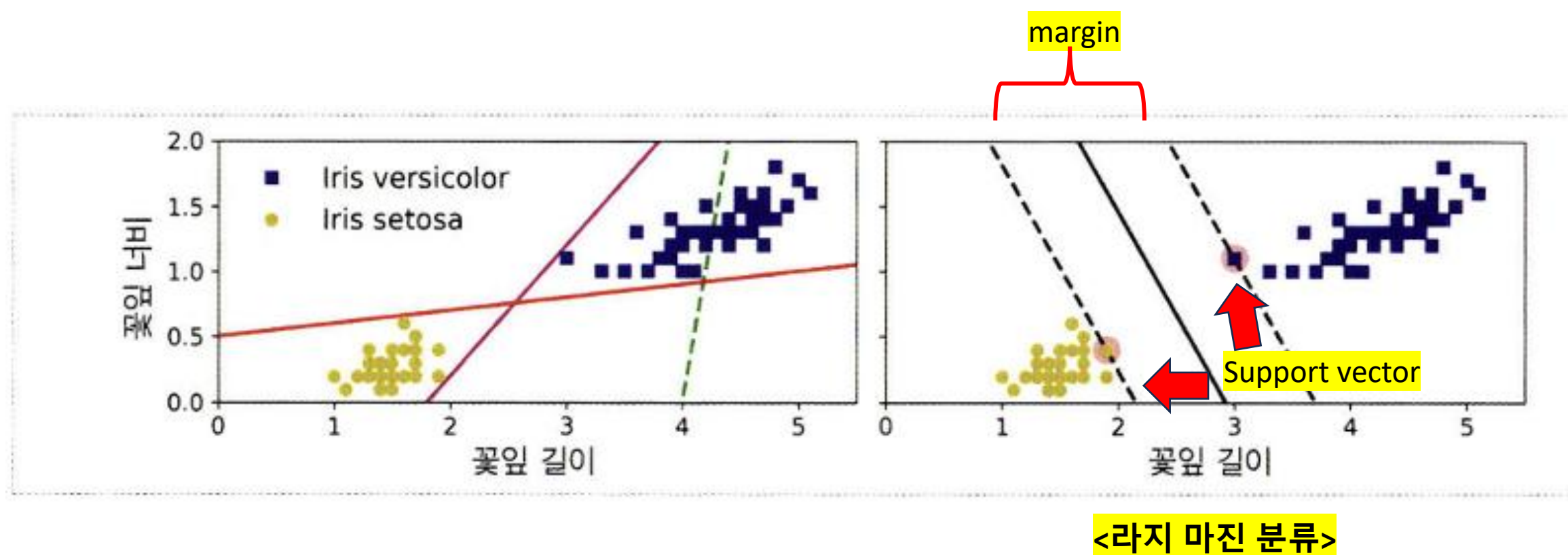
#1.SVM이란

:서포트 벡터 머신(Support vector machine),은 지도형 머신러닝 알고리즘으로,
N차원 공간에서 각 클래스 간의 거리를 최대화하는 최적의 선 또는 *초평면을 찾아
데이터를 분류함

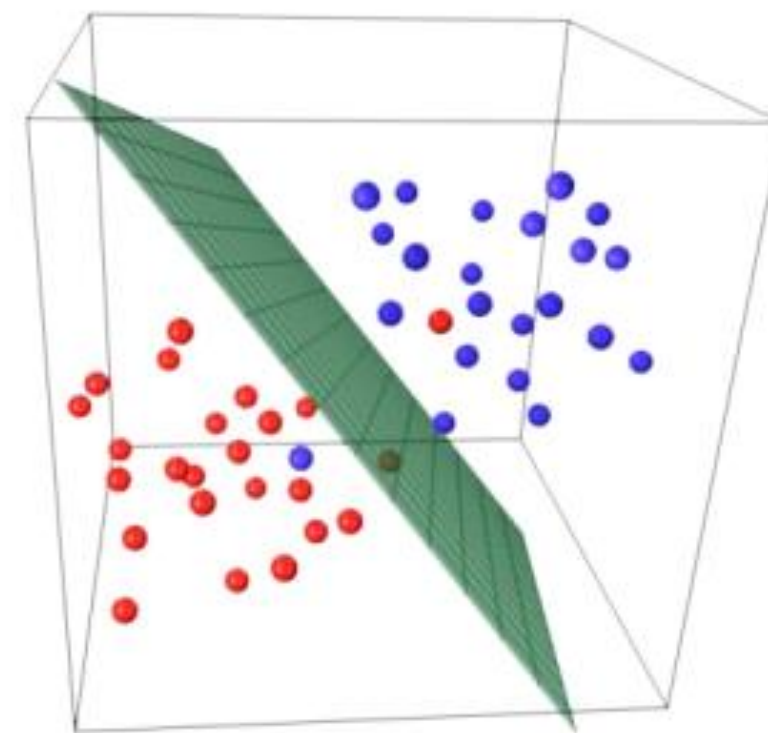
*초평면(hyperplane): N차원 공간을 두부분으로 나누는 N-1차원의 부분 공간

#2.선형 SVM

:선형적으로 분리 가능한 데이터에 사용됨

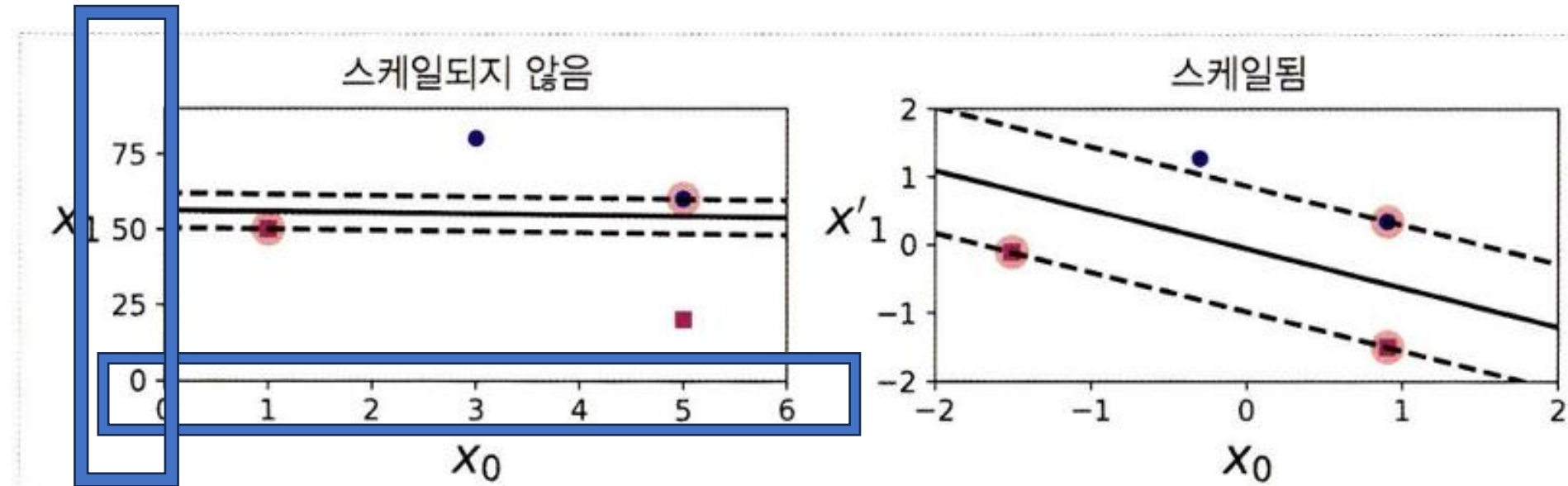


<고차원 초평면>



#5.1 선형 SVM 분류

+) 특정 스케일에 따른 민감성



#2-1. 소프트 마진 분류

*하드 마진 분류

:모든 샘플이 마진 바깥쪽에 올바르게 분류돼 있어야함

문제점

:데이터가 선형적으로 구분돼 있어야함. 이상치에 민감 => 데이터가 선형적인 경우는 드물다

->이러한 한계를 개선하고자 나온 개념 => **소프트 마진 분류**

:어느정도 오분류는 허용

#5.1 선형 SVM 분류

#2-1.소프트 마진 분류

*Slack Variable

:결정경계로부터 잘못 분류된 데이터의 거리를 측정하기 위해 사용
:어느정도 오분류는 허용하는 소프트 마진의 특성을 고려함

Slack Variable 만큼의 초평면 위아래의 오차 허용

*하이퍼 파라미터 C(cost)

:slack variable의 총합에 한계를 줌으로써 조정

c가 작으면

⇒마진 넓음
⇒오류 증가
⇒과소적합의 위험성

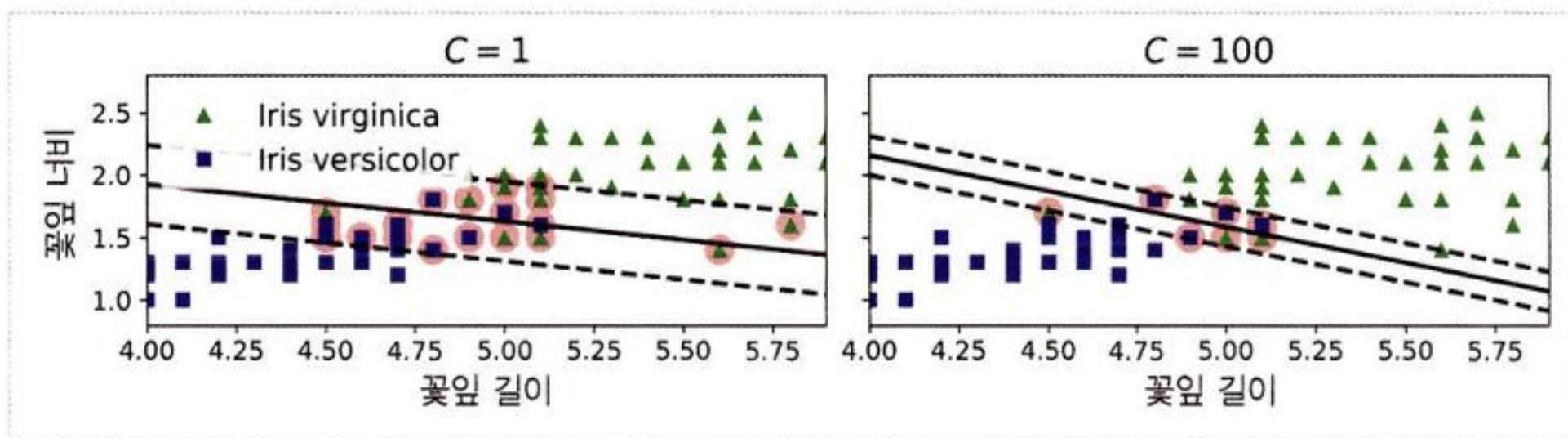
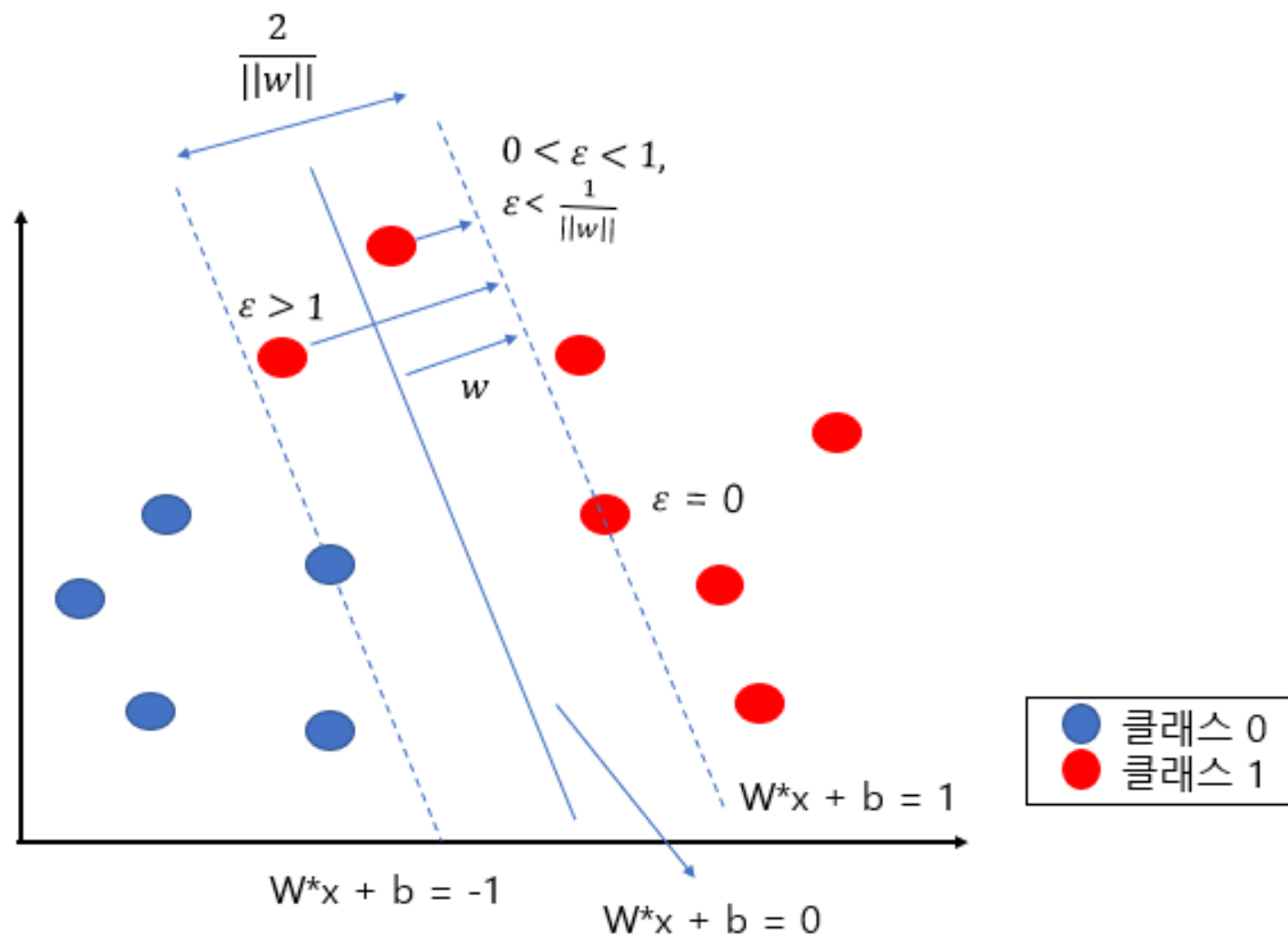


그림 5-4 넓은 마진(왼쪽) 대 적은 마진 오류(오른쪽)



c가 크면

⇒마진 좁음
⇒오류 적음
⇒과적합의 위험성

#5.1 선형 SVM 분류

#2-1.소프트 마진 분류

<붓꽃 데이터셋으로 Iris-Virginia 품종 감지>

```
#선형 svm
import numpy as np
from sklearn import datasets
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = datasets.load_iris()
X = iris['data'][:,(2,3)]      petal length, petal width 두개의 feature만 사용
y = (iris['target']==2).astype(np.float64)
    => 0(setosa), 1(versicolor), 2(virginica) 중에 virginica를 찾음(이진분류)
svm_clf = Pipeline([
    ('scaler', StandardScaler()),
    ('linear_svc', LinearSVC(C=1, loss = 'hinge')),
])
    SVC(kernel = 'linear', C=1) || SGCClassifier(loss = 'hinge', alpha = 1/(m*C)) 사용 가능

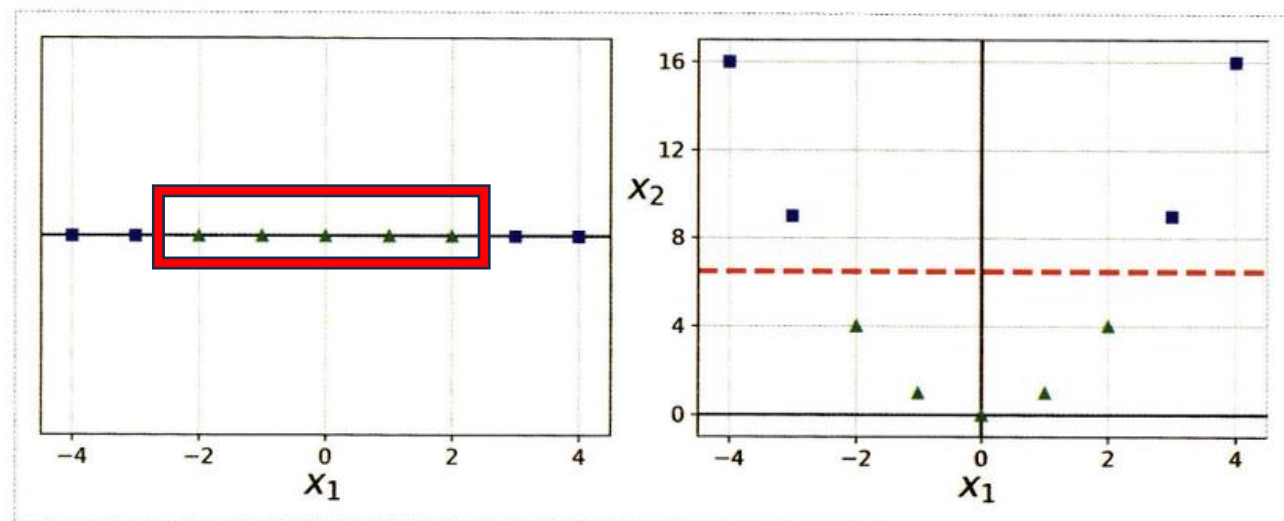
svm_clf.fit(X,y)

svm_clf.predict([[5.5, 1.7]])  petal length = 5.5 // petal_width = 1.7인 샘플 예측

array([1.])      Virginica일 확률 높다고 예측(True = 1, False = 0)
```

#5.2 비선형 SVM 분류

<비선형 데이터셋>



#비선형 svm

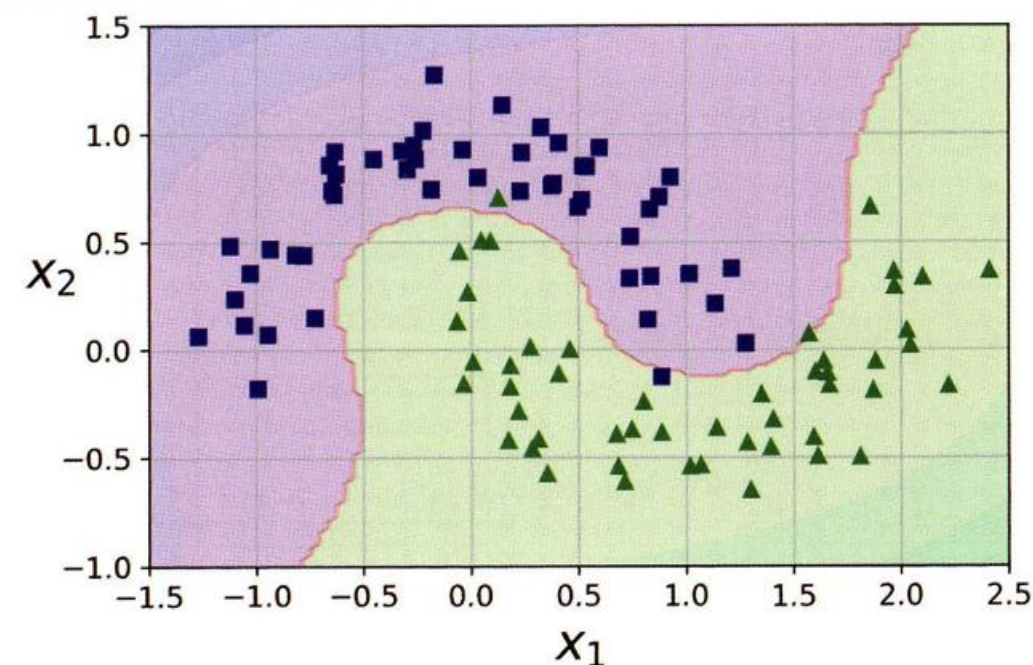
```
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures
```

```
X, y = make_moons(n_samples = 100, noise = 0.15)
```

반달모양(moons)으로 배치된 데이터 형성

```
polynomial_svm_clf = Pipeline([
    ('poly_features', PolynomialFeatures(degree = 3)),
    ('scaler', StandardScaler()),
    ('svm_clf', LinearSVC(C = 10, loss = 'hinge'))
])
```

```
polynomial_svm_clf.fit(X, y)
```

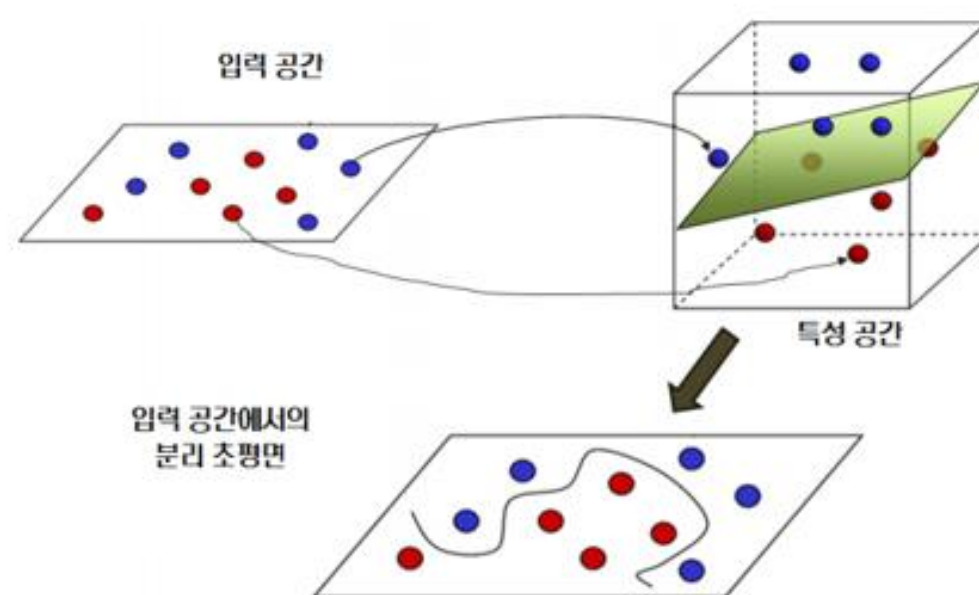


다항식 특성 변환기, degree= 3은 입력 특성을 최대 3차까지 확장한다는 의미

=> x(1차)*x^2(2차) 처럼 고차항과 상호작용 항을 포함시켜, 비선형 데이터를 선형으로 나눌 수 있게 함

#5.2 비선형 SVM 분류

#1. 다항식 커널



고차원 특성 변환 시, 계산이 상당히 복잡해짐
->해결하고자하는 개념 '커널 트릭(kernel trick)'

실제로 특성을 추가하지 않으면서 다항식 특성을 많이 추가할 것 같은 결과를 얻을 수 있음

```
#kernel trick
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
    ('scaler', StandardScaler()),
    ('svm_clf', SVC(kernel = 'poly', degree=3, coef0=1, C=5))
])
poly_kernel_svm_clf.fit(X,y)
```

Coef0 => kernel trick 수식에서의 상수항
작으면(0근처) -> 고차항의 영향이 크다는 걸 의미(비선형적 경계)
크면 -> 낮은 차수 성분이 더 중요(선형적 경계에 가까움)
기본값은 0

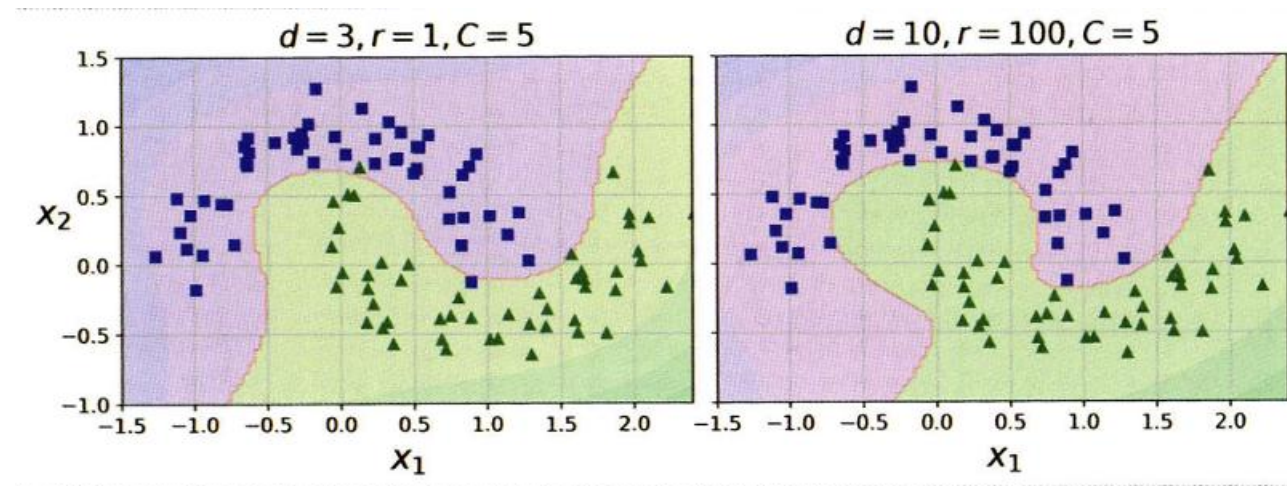


그림 5-7 다항식 커널을 사용한 SVM 분류기

$$\Phi(x) = Ax$$

Mapping function

$$k(x_i, x_j) = \Phi(x_i)^T \Phi(x_j)$$

Kernel trick

#5.2 비선형 SVM 분류

#2. 유사도 특성

식 5-1 가우시안 RBF

$$\phi_{\gamma}(\mathbf{x}, \ell) = \exp\left(-\gamma \|\mathbf{x} - \underline{\ell}\|^2\right)$$

랜드마크 ℓ

0~1의 값을 가짐
랜드마크와 벡터 $\mathbf{x}$ 가 가까우면 => 1에 가까움
멀면 => 0에 가까움

If) n차원 공간에 m개의 데이터(n개의 특성 가진 m개의 샘플)
→ n차원->m차원으로 벡터 변환 가능

Why?
1개의 점 ℓ (랜드마크)를 잡고 모든 점 $\mathbf{x}$ 에 대해 가우시안 RBF 적용
→로부터 모든 $\mathbf{x}$ 의 거리인 m개의 거리 피처를 가지게 됨

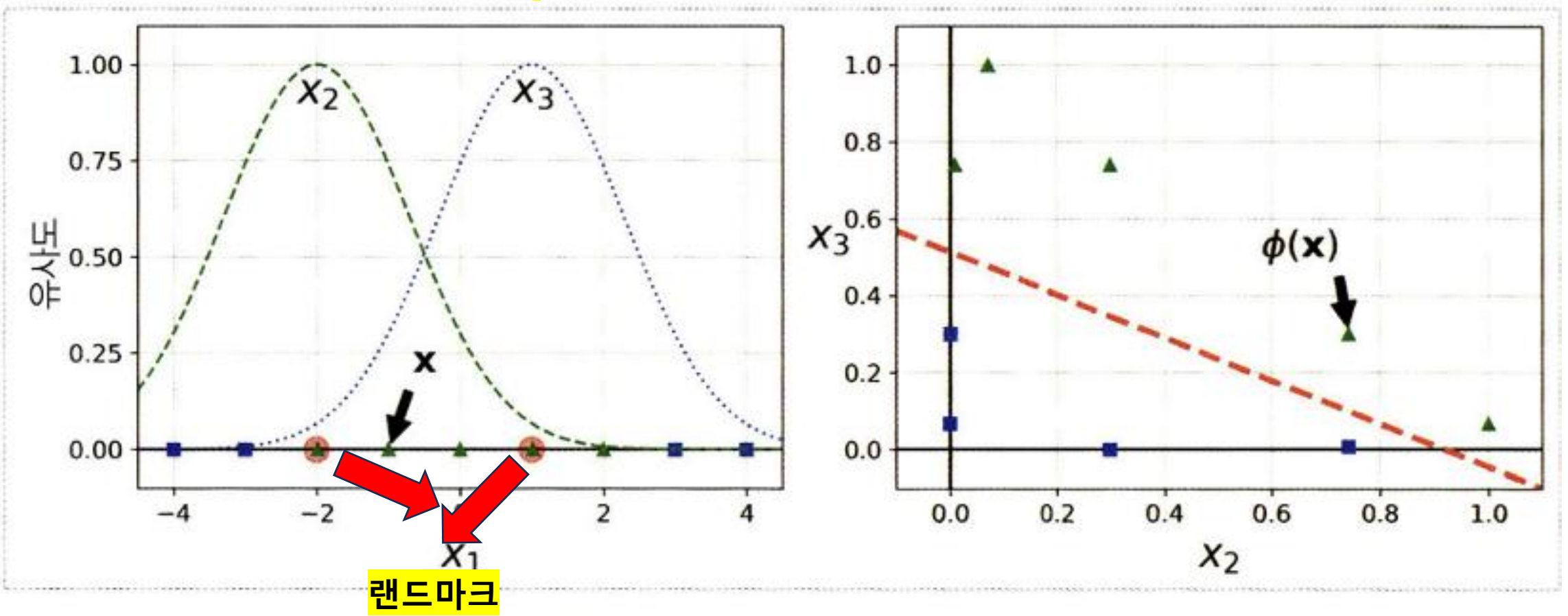


그림 5-8 가우시안 RBF를 사용한 유사도 특성

#5.2 비선형 SVM 분류

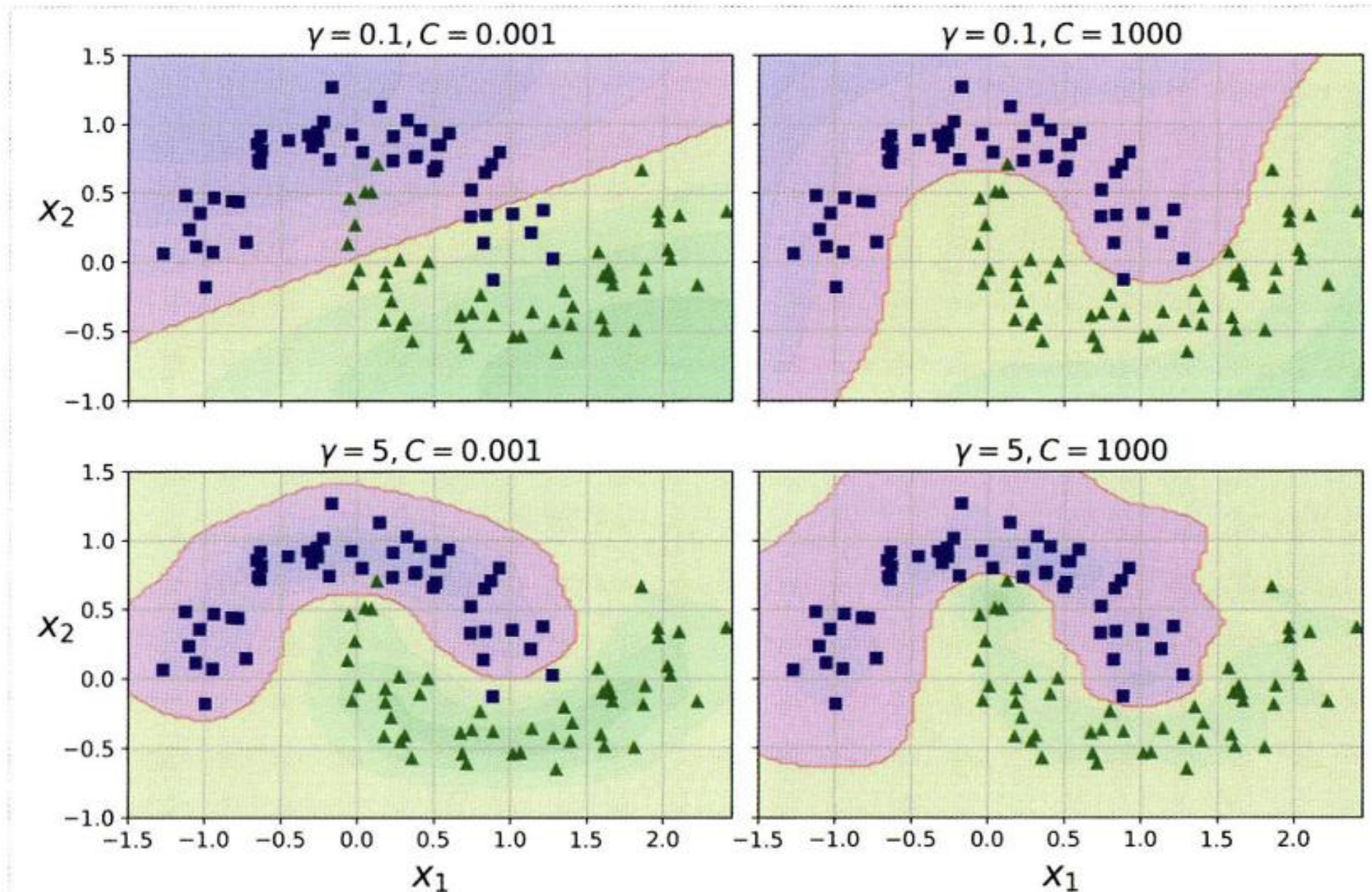
#3.가우시안 RBF커널

#가우시안 RBF 커널

```
rbf_kernel_svm_clf = Pipeline([
    ('scaler', StandardScaler()),
    ('svm_clf', SVC(kernel = 'rbf', gamma = 5, C = 0.001))
])
rbf_kernel_svm_clf.fit(X,y)
```

+ 다른 커널

문자열 커널: 텍스트 문서 or DNA 서열 분류할 때 사용
(문자열 서브시퀀스 커널, 레벤슈타인 거리 기반 커널)



#5.2 비선형 SVM 분류

#4. 계산복잡도

표 5-1 SVM 분류를 위한 사이킷런 파이썬 클래스 비교

파이썬 클래스	시간 복잡도	외부 메모리 학습 자원	스케일 조정의 필요성	커널 트릭
LinearSVC	$O(m \times n)$	아니오	예	아니오
SGDClassifier	$O(m \times n)$	예	예	아니오
SVC	$O(m^2 \times n) \sim O(m^3 \times n)$	아니오	예	예

THANK YOU

